From Gradients to Curvature and Matrix Geometry: A Survey of Optimization Methods for Deep Neural Networks

Faris Nasser and Salah Moussa

***Abstract*—Training a deep neural network involves minimizing a loss function by repeatedly updating its parameters. While gradients indicate how the loss changes, an optimization algorithm determines how this information is used to select each update. Different optimizers address limitations of standard gradient descent by incorporating past gradients, adapting step sizes, accounting for curvature, or exploiting the matrix structure of neural-network parameters. This survey provides a unified overview of these approaches, beginning with the mathematical foundations of optimization and progressing from classical first-order methods to second-order, curvature-aware, and modern matrix-aware algorithms. We explain the principles behind each family, the challenges they aim to address, and the trade-offs they introduce in convergence, computational cost, and memory. By connecting these methods through their treatment of gradient information and parameter-space geometry, the survey provides a framework for understanding their differences and evaluating their suitability for deep neural-network training.**

***Index Terms*—Non-convex optimization, stochastic gradient descent, Adam, Hessian, Newton method, quasi-Newton, natural gradient, K-FAC, Shampoo, Sophia, Muon, preconditioning, neural networks.**

I. Introduction

TRAINING a deep neural network involves finding parameter values that minimize a loss function, which measures how well the network's predictions match the desired outputs. This is achieved through an iterative optimization process in which the model's parameters are repeatedly adjusted to reduce the loss. Gradient descent provides a natural starting point: it uses the gradient to identify a direction of local decrease and a learning rate to determine the size of the update. However, although this approach is conceptually simple, it does not fully address the challenges of optimizing deep neural networks.

These challenges arise from both the structure of the loss function and the practical requirements of training. Deep neural networks typically have non-convex loss landscapes, where curvature can vary considerably across parameter directions. Consequently, a single learning rate may lead to slow progress in some directions and unstable updates in others. Furthermore, computing gradients over an entire training dataset can be expensive, motivating the use of smaller batches that provide faster but noisier gradient estimates. As networks grow, the memory and computational resources required to maintain optimizer statistics or estimate curvature also become increasingly important [1], [2], [3].

These limitations have motivated the development of optimization methods that go beyond directly scaling the current gradient. Momentum-based methods incorporate information from previous updates to improve the consistency of the search direction, while adaptive methods adjust the update magnitude for individual parameters based on historical gradient statistics. Curvature-aware methods use second-order information to account for how the loss changes across directions. More recently, matrix-aware approaches have sought to exploit the structure of neural-network weight matrices to construct more informative updates without necessarily forming the full Hessian [4], [5], [6], [7].

Although these methods share the objective of minimizing training loss, they differ in the information they use, how they transform that information into parameter updates, and the computational resources they require. Understanding these differences is essential for evaluating their advantages and limitations. This survey therefore examines the mathematical principles underlying major optimization methods, tracing their development from conventional gradient descent to adaptive, curvature-aware, and matrix-aware approaches. Particular attention is given to how each family addresses the limitations of simpler methods and the trade-offs involved in doing so.

A. Existing Surveys and Scope

Optimization methods for machine learning have been studied extensively, with existing literature examining different aspects of their mathematical foundations and practical applications. Bottou, Curtis, and Nocedal [3] provide a foundational treatment of optimization for large-scale machine learning, while Nocedal and Wright [1] establish the theoretical principles of classical optimization methods, including Newton, quasi-Newton, and trust-region approaches. Other studies investigate the properties of deep-learning loss landscapes, particularly the role of

F. Nasser and S. Moussa are with the College of Engineering, American University of Sharjah, Sharjah, United Arab Emirates.

Prepared for MLR 570, Advanced Machine Learning, Fall 2026. This is a literature survey; figure slots indicate planned illustrations, not experimental results.

curvature and stationary points in non-convex optimization [8]–[10]. More recent work focuses on developing optimizers that address the computational and memory demands of modern neural-network training [11], [12], [13].

Although these contributions provide valuable insights, they address different aspects of the optimization problem. Classical optimization theory emphasizes mathematical properties and convergence guarantees, while research on modern optimizers often focuses on improving training efficiency in large neural networks. This survey connects these perspectives by examining how optimization methods construct parameter updates, what information they use to guide those updates, and how their mathematical principles relate to their computational requirements. Rather than treating each optimizer as an isolated algorithm, we trace how different approaches address limitations of simpler methods and introduce new trade-offs.

To maintain this focus, the survey considers single-objective, unconstrained optimization of neural networks through empirical risk minimization. It covers deterministic and stochastic first-order methods, approaches that use or approximate curvature information, and modern matrix-aware methods that exploit the structure of neural-network parameters. Related topics, such as learning-rate schedules, normalization, and distributed training, are discussed where they directly affect the comparison of optimization methods. Multi-objective and constrained optimization fall outside the scope of this survey. Finally, the discussion distinguishes theoretical guarantees from empirical findings, with reported performance results attributed to their original studies rather than presented as independently reproduced experiments.

B. Research Questions

As discussed in the previous section, optimization methods differ mainly in how they determine parameter updates. Some improve the use of gradient information, while others introduce additional mathematical information to guide the optimization process. These differences motivate two research questions.

RQ1: How do first-order optimization methods improve upon standard gradient descent?

Standard gradient descent updates model parameters using the current gradient and a learning rate. However, this approach can be inefficient when gradients are noisy or when the loss varies significantly across parameter directions. The first research question examines how first-order methods address these limitations by modifying how gradient information is used. It also considers the additional computational and memory costs of these modifications.

RQ2: How do curvature-aware and matrix-aware methods extend gradient-based optimization?

While first-order methods focus on improving the use of gradients, other approaches seek to construct better updates by considering the curvature of the loss function or the structure of model parameters. The second research question explores how these methods use such information and whether the resulting improvements justify their additional computational cost.

Together, these questions guide the comparison of optimization methods throughout the survey. The objective is not to identify a single best optimizer, but to understand why different approaches are developed and under what conditions their advantages become meaningful.

C. Contributions and Organization

This survey brings together the mathematical foundations of neural-network optimization and the principles underlying different optimization methods. Rather than introducing a new algorithm, its main contribution is to explain how existing approaches are related and why their differences matter.

The discussion begins by establishing the mathematical concepts needed to understand optimizer behavior. These foundations are then used to examine how optimization methods construct parameter updates. A common mathematical framework helps connect several of these methods, while also making clear where their underlying principles differ. Throughout the survey, attention is given to the relationship between an optimizer's mathematical design and its practical cost. This provides a consistent basis for comparing theoretical guarantees with findings reported in the literature.

The remainder of the paper follows this progression. Section II introduces the mathematical preliminaries needed to understand gradient-based optimization. Section III examines first-order methods and addresses RQ1. Section IV develops the role of curvature in optimization, while Section V considers structured and matrix-aware approaches. Together, these sections address RQ2. Section VI compares the methods and discusses their limitations. Section VII identifies open research directions, and Section VIII concludes the survey. Figure 1 provides an overview of the methods covered and their relationships.

II. Mathematical Preliminaries

A. From Data to a Non-Convex Objective

Training a neural network involves finding the model parameters that minimize a loss function. Given a dataset of n samples, $\{(x_i, y_i)\}_{i=1}^{n}$, the training objective can be expressed as

$$L(\theta) = \frac{1}{n}\sum_{i=1}^{n} \ell(f_\theta(x_i), y_i), \qquad \theta \in \mathbb{R}^d. \tag{1}$$

Here, $f_\theta(x_i)$ represents the model's prediction for input x_i, while ℓ measures the difference between this prediction and the corresponding target y_i. The parameter vector θ contains the model's trainable weights and biases. The objective $L(\theta)$ is therefore the average training loss that the optimizer seeks to minimize.

The difficulty of this optimization problem depends partly on the model's structure. For a linear model with squared-error loss, the objective is a convex quadratic function. This provides a relatively simple optimization

setting, since every local minimum is also a global minimum. Deep neural networks, however, involve multiple layers whose parameters interact. Even without nonlinear activation functions, multiplying the weight matrices of different layers can produce a non-convex objective. As a result, the loss landscape may contain saddle points and local minima, making it difficult to guarantee that an optimization method will find the global minimum [14], [8].

It is also important to distinguish minimizing the training loss from achieving good performance on unseen data. Optimization focuses on reducing the objective defined by the training samples, while generalization concerns how well the trained model performs on new examples. A model can achieve a low training loss without necessarily generalizing well [15], [16], [17]. This distinction matters when evaluating optimizers, since faster loss reduction does not always translate into better predictive performance.

B. Gradient, Hessian, and Local Taylor Models

Having defined the training objective, the next step is to determine how its value changes as the model parameters are updated. The gradient provides this information by measuring the sensitivity of the loss to each parameter. For a differentiable objective $L(\theta)$, the gradient is defined as

$$g(\theta) = \nabla L(\theta), \qquad g_j = \frac{\partial L}{\partial \theta_j}.$$

Each component indicates how the loss changes locally with respect to one parameter. For a small parameter update Δ, this relationship can be approximated using the first-order Taylor expansion:

$$L(\theta + \Delta) \approx L(\theta) + g^\top \Delta. \tag{2}$$

The term $g^\top \Delta$ estimates the change in loss caused by the update. To reduce the loss, we therefore seek a direction for which this term is negative. By the Cauchy–Schwarz inequality,

$$g^\top \Delta \geq -\|g\|_2 \|\Delta\|_2,$$

the direction opposite to the gradient produces the greatest predicted decrease among updates of equal Euclidean length. This explains the basic principle of gradient descent. However, this prediction is based on a local linear approximation and does not guarantee that an arbitrarily large step will reduce the actual loss.

The limitation of this approximation becomes important when the slope changes significantly along the update direction. To describe this behavior, we introduce the Hessian matrix, which contains the second derivatives of the loss:

$$H(\theta) = \nabla^2 L(\theta), \qquad H_{jk} = \frac{\partial^2 L}{\partial \theta_j \partial \theta_k}.$$

The Hessian captures local curvature and allows the first-order approximation to be extended into a quadratic model:

$$L(\theta + \Delta) \approx L(\theta) + g^\top \Delta + \frac{1}{2} \Delta^\top H \Delta. \tag{3}$$

The additional term accounts for how the gradient changes as the parameters move away from their current values. Its behavior depends on the curvature along the chosen direction. If v is a unit eigenvector of the Hessian with eigenvalue λ, an update of the form $\Delta = av$ contributes $\frac{1}{2}\lambda a^2$ to the quadratic term. Positive eigenvalues indicate upward curvature, while negative eigenvalues indicate downward curvature along the corresponding directions.

This distinction is particularly important near stationary points, where the gradient is zero. Such a point is not necessarily a minimum. If the Hessian has a negative eigenvalue, there is a direction of negative curvature, which rules out a local minimum. In contrast, a positive semidefinite Hessian is a necessary but not sufficient condition for a local minimum at a twice-differentiable stationary point [1], [8].

The gradient and Hessian therefore provide complementary information. The gradient indicates the direction of local change, while the Hessian describes how that direction may change as optimization proceeds. However, both Taylor models are local approximations. Understanding when they provide reliable guidance requires additional assumptions about how rapidly the gradient can vary. This motivates the smoothness conditions introduced in the next subsection.

C. Smoothness, the Descent Lemma, and Safe Steps

The previous subsection showed that moving against the gradient produces the greatest predicted decrease in loss for a sufficiently small update of fixed length. However, this prediction comes from a local approximation. As the parameters move, the gradient may change, causing the actual loss to behave differently from what the approximation suggests. To understand when a gradient-based update is reliable, we need to account for how quickly the gradient can vary.

This behavior can be described through *smoothness*. A differentiable loss function is said to be β-smooth over a region if

$$\|\nabla L(x) - \nabla L(y)\|_2 \leq \beta \|x - y\|_2. \tag{4}$$

The constant β limits how rapidly the gradient can change between two points. For a twice-differentiable function, this condition holds when the spectral norm of the Hessian is bounded by β throughout the region. A larger value of β allows greater changes in the gradient and therefore indicates that more caution may be needed when selecting an update.

Smoothness leads to an important result known as the *descent lemma*. For an update Δ, it provides the following upper bound on the loss:

$$L(x + \Delta) \leq L(x) + \nabla L(x)^\top \Delta + \frac{\beta}{2} \|\Delta\|_2^2. \tag{5}$$

Unlike the first-order Taylor approximation in Eq. (2), this expression provides a bound on the actual loss under the smoothness assumption. The first two terms describe the predicted change based on the current gradient. The final

term accounts for the possible effect of curvature as the parameters move away from their current values.

We can now use this result to examine gradient descent. Substituting the update $\Delta = -\eta \nabla L(x)$ into Eq. (5) gives

$$L(x - \eta g) \leq L(x) - \eta \|g\|_2^2 + \frac{\beta \eta^2}{2} \|g\|_2^2, \tag{6}$$

which simplifies to

$$L(x - \eta g) \leq L(x) - \eta \left(1 - \frac{\beta \eta}{2}\right) \|g\|_2^2. \tag{7}$$

This bound explains the relationship between the learning rate and the curvature of the loss landscape. When $0 < \eta < 2/\beta$, the expression guarantees a decrease in loss whenever the gradient is nonzero, provided the smoothness condition holds along the update. Choosing $\eta \leq 1/\beta$ gives a more conservative bound on this decrease.

The result also highlights a limitation of using a fixed learning rate. A step size that is suitable in a relatively flat region may be too large in a region where the loss changes rapidly. This becomes particularly important when the loss has different curvature along different parameter directions. The next subsection examines this difficulty and explains how it affects the behavior of gradient descent.

D. Curvature, Conditioning, and Rotated Valleys

The previous subsection established that the learning rate must be chosen carefully to ensure a decrease in loss. However, the curvature of a loss function is not necessarily the same in every direction. A learning rate that allows stable updates along a steep direction may produce very slow progress along a flatter one. This difficulty can be illustrated using the quadratic function

$$L(x, y) = \frac{1}{2}(100x^2 + y^2), \qquad H = \begin{bmatrix} 100 & 0 \\ 0 & 1 \end{bmatrix}. \tag{8}$$

The Hessian shows that the curvature in the x-direction is one hundred times greater than in the y-direction. Consequently, the loss forms a narrow valley, as illustrated in Fig. 2(b). Gradient descent must use a learning rate small enough to avoid unstable movement across the steep direction. However, this restriction also limits progress along the flatter direction.

To see this, consider gradient descent with a learning rate of $\eta = 0.01$. Since the gradient is $g = (100x, y)^\top$, the parameter updates become

$$x_{t+1} = (1 - 100\eta)x_t, \qquad y_{t+1} = (1 - \eta)y_t.$$

With the chosen learning rate, the x-component reaches zero in one step, while the y-component retains 99% of its previous value. Although the steep direction is corrected immediately in this example, progress along the flatter direction remains slow. This difference is captured by the *condition number*, defined for a positive-definite Hessian as the ratio of its largest to smallest eigenvalues:

$$\kappa = \frac{\lambda_{\max}}{\lambda_{\min}} = 100.$$

A large condition number indicates that the loss has substantially different curvature across directions. Such problems are described as *ill-conditioned*, and they can slow the convergence of gradient descent [18], [1].

The difficulty becomes more subtle when the valley is rotated relative to the parameter coordinates, as shown in Fig. 2(c). Mathematically, this corresponds to replacing the Hessian with $Q^\top H Q$, where Q is an orthogonal rotation matrix. This transformation preserves the eigenvalues and therefore leaves the condition number unchanged. However, it introduces off-diagonal terms into the Hessian, meaning that changes in one parameter can influence the gradient associated with another.

This distinction matters when designing optimization methods. A diagonal scaling method can adjust the update magnitude of each parameter independently, which may help when the main curvature directions align with the parameter coordinates. In a rotated valley, however, these directions no longer align. Independent scaling cannot generally account for the interactions between parameters. Full-matrix methods can represent these interactions, although doing so requires additional computation and memory.

These examples show that optimization difficulty depends not only on the amount of curvature but also on how that curvature is distributed across parameter directions. They also motivate methods that use more information than a single learning rate or independent parameter scaling. Before examining those methods, we next consider how curvature affects the stability of optimization near stationary points.

E. Local Linear Stability and Non-Convex Stationarity

The previous subsection showed how differences in curvature can affect the progress of gradient descent. Curvature also plays an important role in determining whether the optimization process is stable. In particular, it helps explain what happens when the parameters approach a stationary point, where the gradient becomes zero.

Consider a stationary point θ^* satisfying $\nabla L(\theta^*) = 0$. Let $e_t = \theta_t - \theta^*$ represent the difference between the current parameters and this point. Near θ^*, the gradient can be approximated using the Hessian $H^* = \nabla^2 L(\theta^*)$:

$$\nabla L(\theta_t) \approx H^* e_t.$$

Substituting this approximation into the gradient descent update gives

$$e_{t+1} \approx (I - \eta H^*) e_t. \tag{9}$$

This expression describes how the distance from a stationary point changes after an update. To understand its behavior, consider an eigenvector of the Hessian with eigenvalue λ. Along this direction, the error is multiplied by $1 - \eta\lambda$ at each step.

When $0 < \eta\lambda < 2$, the magnitude of this multiplier is less than one, so the error decreases. This corresponds to stable movement toward the stationary point along that direction. If $\lambda < 0$, the error instead grows, causing the

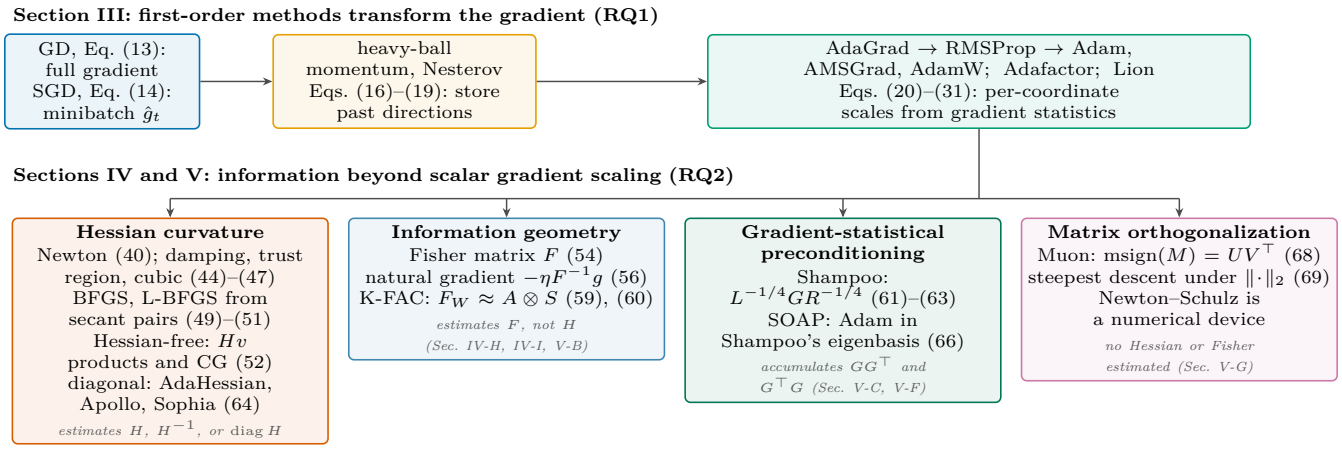


Fig. 1: Organization of the survey. The branches separate Hessian curvature, information geometry, gradient-statistical preconditioning, and matrix orthogonalization; their similar algebra does not make them the same approximation.

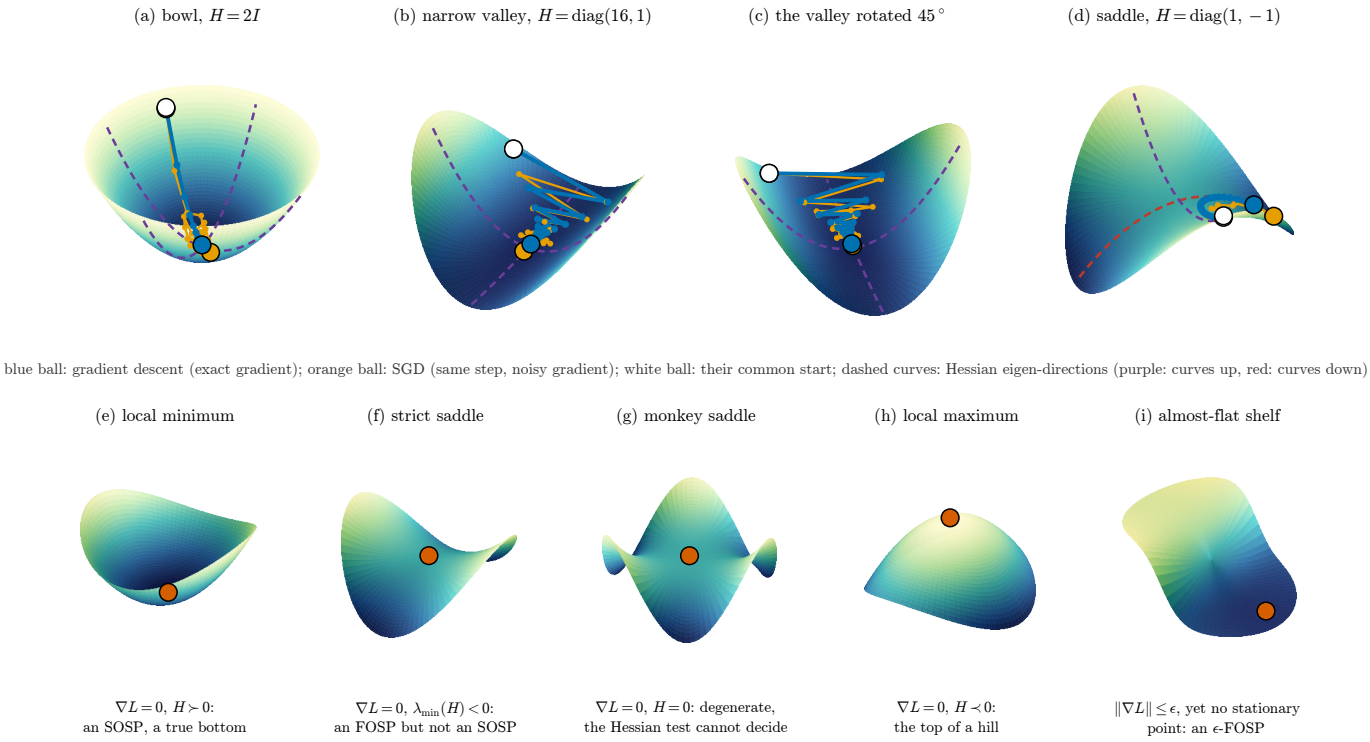


Fig. 2: Loss geometry in two dimensions. Gradients indicate local slope; Hessian eigenvectors indicate directions of principal curvature. The bowl and saddle are local models, not claims about global network landscapes.

iterates to move away from the stationary point. Such behavior can be useful near a saddle point, where negative curvature indicates a direction of descent. However, even with positive curvature, an excessively large learning rate can cause instability. When $\eta\lambda > 2$, the error grows while alternating in sign, producing increasingly large oscillations. These behaviors help explain the different trajectories illustrated in Fig. 2.

This analysis also shows why a small or vanishing gradient does not necessarily indicate that optimization has reached a minimum. As illustrated in Fig. 2(e)–(h), stationary points can have different local geometries. A first-order stationary point is commonly identified by

$$\|\nabla L(\theta)\|_2 \le \epsilon,$$

where ϵ is a small tolerance. To account for curvature, an approximate second-order stationary point additionally satisfies

$$\lambda_{\min}(H(\theta)) \ge -\gamma,$$

where γ limits the amount of negative curvature. This additional condition helps distinguish points that are approximately locally minimizing from those with a signifi-

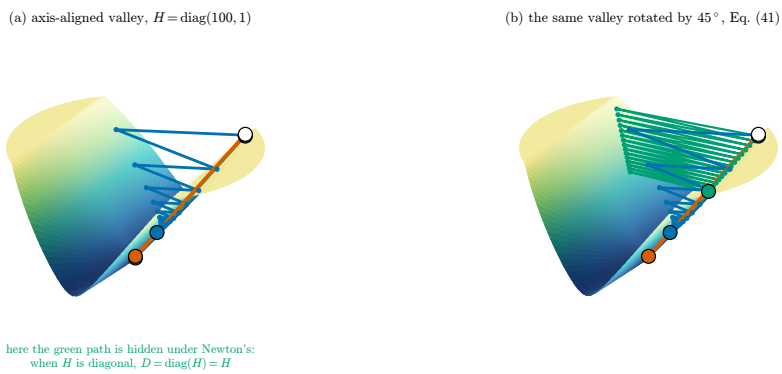


Fig. 3: Why conditioning and coordinate choice matter. The Hessian eigenvalues are unchanged by rotation, but diagonal rescaling is tied to the chosen coordinates; full-matrix transformations can follow rotated principal axes.

cant direction of negative curvature. It does not, however, guarantee a global minimum.

These results clarify what convergence means in non-convex optimization. Gradient-based methods are often analyzed in terms of reaching a point with a sufficiently small gradient rather than finding the globally best solution. Stronger guarantees, such as escaping saddle points, require additional assumptions or algorithmic mechanisms [9], [10], [19].

So far, this analysis has assumed that the optimizer uses the exact gradient. In practice, neural networks are often trained using minibatches, which introduce uncertainty into the estimated gradient. The next subsection examines how this stochasticity changes the update process and its convergence behavior.

F. Stochastic Gradients: What Minibatches Change

The previous analysis considered gradient descent using the exact gradient of the training loss. Computing this gradient requires evaluating every training sample before updating the model parameters. For large datasets, this can make each update expensive. Stochastic gradient descent (SGD) addresses this limitation by estimating the gradient using a smaller, randomly selected subset of the training data, known as a *minibatch*.

For a minibatch B_t, the estimated gradient at iteration t is

$$\hat{g}_t = \frac{1}{|B_t|} \sum_{i \in B_t} \nabla L_i(\theta_t) = g_t + \xi_t. \tag{10}$$

Here, g_t is the full gradient, while ξ_t represents the error introduced by estimating it from a minibatch. Under appropriate random sampling, this error has zero expected value, meaning that the minibatch gradient is an unbiased estimate of the full gradient. Nevertheless, individual estimates may differ considerably from the true gradient, particularly when the batch size is small.

This introduces a trade-off between computational efficiency and gradient accuracy. Smaller minibatches reduce the amount of data processed per update but generally produce noisier estimates. Larger minibatches provide more stable estimates at a greater computational cost per update. Under independent sampling, the variance of the gradient estimate decreases proportionally to the inverse of the batch size. However, the effect on overall training efficiency also depends on how well the computation can be parallelized [20], [21], [2], [22], [23].

The uncertainty introduced by minibatches also affects convergence. For ordinary gradient descent, smoothness provides conditions under which a sufficiently small learning rate guarantees a decrease in loss. With stochastic gradients, this guarantee becomes more limited because the estimated direction may differ from the true direction of descent. The expected change in loss therefore depends on both the progress predicted by the full gradient and the additional error caused by minibatch sampling.

$$\mathbb{E}_t[L_{t+1}] \leq L_t - \eta(1 - \tfrac{\beta\eta}{2})\|g_t\|^2 + \frac{\beta\eta^2\sigma^2}{2|B_t|}. \tag{11}$$

As training approaches a stationary point, the true gradient becomes smaller, but stochastic fluctuations may remain significant.

These observations explain why the behavior of SGD depends on both the learning rate and the batch size. They also motivate optimization methods that use information from previous gradients to reduce the effect of noisy updates. Before examining those methods, we introduce a common framework for understanding how different optimizers transform gradient information into parameter updates.

G. A Unifying Variable-Metric Model—and Its Limits

So far, we have examined how gradients describe the local behavior of the loss function and how minibatch sampling affects their accuracy. However, even when the gradient is known, the optimizer must still determine how the parameters should be updated. Standard gradient descent uses the negative gradient and scales it by a learning rate. More advanced methods modify this direction to account for differences in how the loss behaves across parameter directions.

TABLE I: The information used by representative optimizer families. "Curvature" is not a synonym for every matrix-valued update.

Family	Constructed quantity	Geometric interpretation
SGD / momentum	Gradient, past gradients	Euclidean update with history
Adam / Adafactor	Squared-gradient statistics	Coordinate-dependent scaling
Newton / BFGS	Hessian or secant approximation	Local quadratic curvature
Natural gradient / K-FAC	Fisher-information geometry	Distributional displacement
Shampoo / SOAP	Structured gradient statistics	Tensor-coordinate preconditioning
Muon	Orthogonalized momentum matrix	Spectral-norm steepest direction

A useful way to understand these methods is through a *variable-metric framework*. Rather than treating movement along every parameter direction equally, this approach uses a matrix M_t to control how strongly the optimizer moves in different directions. When M_t is positive definite, the resulting update can be written as

$$\Delta_t = -\eta M_t^{-1} g_t, \qquad \theta_{t+1} = \theta_t + \Delta_t. \tag{12}$$

Here, g_t is the gradient at the current parameters, η is the learning rate, and Δ_t is the proposed parameter change. The matrix M_t determines how the gradient is transformed before the update is applied. This process is commonly referred to as *preconditioning*.

The framework connects several optimization methods through their choice of M_t. Standard gradient descent corresponds to the identity matrix, which leaves the gradient direction unchanged. Adaptive methods use diagonal scaling to adjust individual parameter updates. Newton's method instead uses the Hessian to account for local curvature, provided the resulting matrix is suitable for defining a descent direction. Natural-gradient methods use the Fisher information matrix to describe how parameter changes affect the model's predicted distributions.

Although these methods can be expressed through similar update rules, the matrices they use represent different information. In particular, the gradient statistics used by Adam should not be interpreted as direct estimates of the Hessian. The same distinction applies to modern matrix-aware methods. For example, Muon modifies the geometry of matrix updates through orthogonalization rather than estimating curvature, so it is not generally described by the quadratic metric framework above.

The value of this framework is therefore not that all optimizers are mathematically equivalent. Rather, it provides a common starting point for understanding how different methods modify the gradient and why those modifications matter. Table I summarizes the information used by the main optimizer families discussed in this survey. The next subsection considers the computational costs of these choices, which become increasingly important as neural networks grow larger.

H. What Counts as Cost or Convergence?

The previous subsection showed how optimization methods can improve parameter updates by using additional information about the loss function. However, obtaining and processing this information comes at a cost. An optimizer that makes greater progress in each iteration may still be less efficient if its updates require substantially more computation or memory. Evaluating optimization methods therefore requires considering both their convergence behavior and the resources needed to achieve it.

Memory requirements are particularly important for large neural networks. For a model with d parameters, storing the full Hessian requires $O(d^2)$ values. This becomes impractical as the number of parameters grows. Structured methods address this limitation by approximating the information needed for an update using smaller matrices or compact statistics. For example, a weight matrix of size $m \times n$ would require m^2n^2 values for a full layer-wise curvature matrix, while separate row and column factors require only $m^2 + n^2$. These reductions can make more advanced optimization methods practical, although their actual memory consumption also depends on implementation details [5], [6], [24], [25].

Computational cost introduces a similar distinction. Some methods rely mainly on gradient evaluations, while others require additional matrix operations or iterative calculations. As a result, fewer optimization steps do not necessarily imply faster training. A meaningful comparison must consider the total computation required to reach a given level of performance, rather than iteration count alone.

Convergence guarantees must also be interpreted carefully. Different theoretical results may measure different forms of progress. For example, some results describe how quickly the loss approaches its minimum under convexity assumptions. Others measure how quickly the gradient becomes small in a non-convex problem. These guarantees cannot be compared directly because they describe different outcomes and may rely on different assumptions [18], [3], [26].

These considerations provide the basis for evaluating the methods discussed in the remainder of the survey. Their effectiveness depends not only on how they construct updates, but also on the resources they consume and the conditions under which their convergence properties hold. With these foundations established, Section III begins with gradient descent and examines how first-order optimization methods improve its behavior.

III. First-Order Optimization: From Gradients to Adaptivity (RQ1)

A. Gradient Descent and Stochastic Gradient Descent

Gradient descent (GD) is one of the simplest methods for minimizing a differentiable loss function. It repeatedly

adjusts the model parameters in the direction opposite to the gradient, where the loss is expected to decrease. The size of each update is controlled by a learning rate η_t. At iteration t, the update is given by

$$\theta_{t+1} = \theta_t - \eta_t \nabla L(\theta_t). \tag{13}$$

The negative gradient determines the direction of the update, while the learning rate determines how far the parameters move. If the learning rate is too small, optimization may progress slowly. If it is too large, the updates may overshoot regions of low loss or become unstable. As discussed in Section II, this behavior is closely related to the curvature of the loss landscape.

A practical limitation of GD is that every update requires computing the gradient over the entire training dataset. This can be expensive when the dataset is large. Stochastic gradient descent (SGD) addresses this problem by replacing the full gradient with an estimate computed from a randomly selected minibatch:

$$\theta_{t+1} = \theta_t - \eta_t \hat{g}_t. \tag{14}$$

Here, $\hat{g}_t$ is the minibatch gradient introduced in Eq. (10). Since each update processes only part of the dataset, SGD can update the model more frequently for a given amount of data processed. This makes it particularly suitable for large-scale neural-network training [20], [21].

The computational advantage of SGD comes with additional uncertainty. Because different minibatches may produce different gradient estimates, successive updates can fluctuate even when the parameters are close to a stationary point. This noise can make the optimization trajectory less stable than that of full-batch GD. Nevertheless, minibatch training is often more practical because it reduces the work required for individual updates and can benefit from parallel computation [2], [3].

An important question is whether these methods eventually converge. For a sufficiently smooth loss function that is bounded below, GD with an appropriate learning rate can be shown to make progress toward a stationary point.

$$\boxed{\min_{0 \le t < T} \|g_t\|^2 \le \frac{2[L(\theta_0) - L_*]}{\eta T}.} \tag{15}$$

In non-convex optimization, this generally means reaching a region where the gradient is small, rather than guaranteeing that the global minimum has been found. SGD requires additional assumptions about the variability of its gradient estimates, since stochastic noise affects the expected progress of each update [3], [26], [27].

Although GD and SGD differ in how they obtain gradients, both construct their updates using the current gradient alone. Neither explicitly accounts for whether previous updates have repeatedly pointed in a similar direction. This limitation motivates momentum-based methods, which use information from earlier iterations to guide subsequent updates.

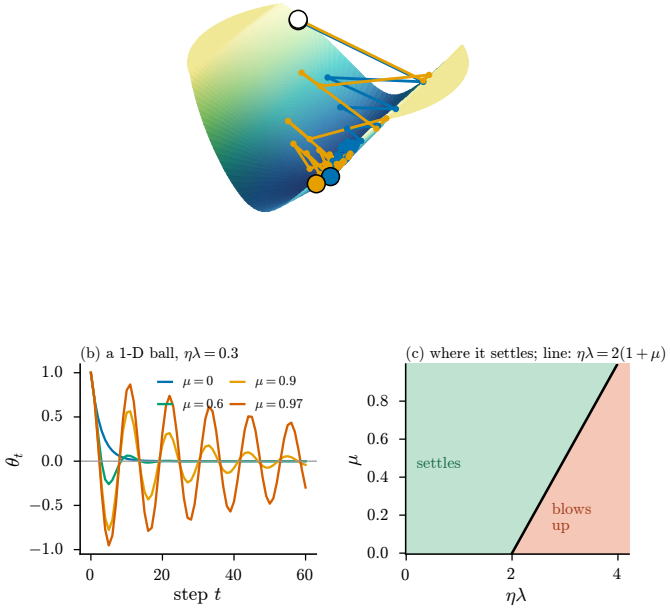


Fig. 4: Momentum integrates directions over time. Across a narrow valley, alternating components may cancel, but excessive momentum can create long-lived oscillations or instability.

B. Momentum: Why Store Past Directions?

Gradient descent determines each update using the current gradient without explicitly accounting for previous directions. This can lead to slow progress when the loss landscape contains narrow valleys. As illustrated in Fig. 4(a), the gradient may repeatedly point toward opposite sides of a valley, causing the optimization trajectory to oscillate rather than move efficiently toward the minimum.

Momentum-based optimization addresses this problem by maintaining a running combination of past gradients. Instead of responding only to the current gradient, the optimizer uses this accumulated information to determine its next update. Polyak's heavy-ball method expresses this idea as

$$v_t = \mu v_{t-1} + g_t, \tag{16}$$

$$\theta_{t+1} = \theta_t - \eta v_t, \qquad 0 \le \mu < 1. \tag{17}$$

Here, v_t represents the accumulated gradient direction, while μ controls how strongly previous gradients influence the current update. When $\mu = 0$, the method reduces to ordinary gradient descent. As μ increases, the optimizer retains more information from earlier iterations.

The benefit of momentum becomes clear when considering how gradient directions change over time. If successive gradients point in similar directions, their contributions accumulate and encourage continued movement. If they repeatedly point in opposite directions, their contributions

partially cancel. In a narrow valley, this can reduce movement across the steep sides while maintaining progress along the valley. Figure 4(a) illustrates this behavior by comparing SGD and momentum on the same loss landscape.

However, retaining past directions also introduces a risk. When the optimizer approaches a minimum, its accumulated momentum may continue pushing the parameters beyond that point. This can produce oscillations or even instability if the learning rate and momentum coefficient are too large. Figures 4(b) and 4(c) illustrate how these two settings influence the resulting trajectory and its stability.

Momentum therefore improves gradient descent by making updates less sensitive to changes in the current gradient direction. Its effectiveness depends on how much historical information is retained and how strongly that information influences each step. This motivates a related approach, Nesterov acceleration, which modifies momentum by evaluating the gradient at a point ahead of the current parameters.

C. Nesterov Acceleration: A Gradient at the Look-Ahead Point

Momentum improves gradient descent by incorporating past gradient information into the current update. However, because the accumulated direction can continue pushing the parameters forward, the optimizer may overshoot a minimum. Nesterov acceleration modifies this approach by evaluating the gradient at a point ahead of the current parameters, allowing the update to account for the slope at the anticipated position.

One commonly used formulation of Nesterov acceleration is

$$y_t = \theta_t + \mu(\theta_t - \theta_{t-1}), \tag{18}$$

$$\theta_{t+1} = y_t - \eta \nabla L(y_t). \tag{19}$$

The first equation defines a look-ahead point y_t based on the previous parameter movement. The second evaluates the gradient at this point and uses it to determine the next parameters. Unlike standard momentum, which combines the current gradient with an accumulated direction, Nesterov acceleration incorporates the effect of previous movement before evaluating the new gradient.

The motivation is that the slope at the look-ahead point may provide more useful information about the direction in which the optimizer is already moving. For example, when approaching the bottom of a valley, the gradient at this anticipated position can reveal a change in slope that is not yet apparent at the current parameters. This can help adjust the update before the accumulated movement carries the parameters too far.

Nesterov acceleration also has important theoretical properties. For smooth convex objectives, suitable versions achieve faster convergence rates than ordinary gradient descent [28], [18]. However, these guarantees depend on the objective and the choice of algorithm parameters. They do not automatically extend to general non-convex neural-network training.

Both momentum and Nesterov acceleration remain first-order methods because they rely on gradients rather than explicit curvature information. Their main contribution is to improve how previous movement influences the current update. However, neither directly addresses the possibility that different parameters may require different update magnitudes. This motivates adaptive optimization methods, beginning with AdaGrad.

D. AdaGrad: Learning Rates for Different Coordinates

Momentum-based methods improve gradient descent by using past gradients to guide the update direction. However, they still rely on a shared learning rate for all parameters. This can be inefficient when different parameters experience gradients of very different magnitudes. AdaGrad addresses this limitation by adjusting the effective learning rate of each parameter individually.

Rather than using the same scaling throughout training, AdaGrad accumulates the squared gradients associated with each parameter. The update is defined as

$$s_{t,j} = s_{t-1,j} + \hat{g}_{t,j}^2, \tag{20}$$

$$\theta_{t+1,j} = \theta_{t,j} - \frac{\eta}{\sqrt{s_{t,j} + \varepsilon}} \hat{g}_{t,j}. \tag{21}$$

Here, $s_{t,j}$ stores the accumulated squared gradients for parameter j, while ε is a small constant used to prevent numerical instability. As the accumulated value increases, the effective learning rate for that parameter decreases.

This mechanism allows AdaGrad to respond differently to parameters with different gradient histories. Parameters that consistently receive large gradients are assigned smaller updates over time. In contrast, parameters with infrequent or small gradients retain relatively larger effective learning rates. This behavior is particularly useful when training models with sparse features, where some parameters are updated much less frequently than others [29].

However, AdaGrad's main limitation follows directly from how it accumulates gradient information. Since squared gradients are continuously added, the accumulated values can only increase. As training continues, the effective learning rates may therefore become excessively small, slowing further progress. Earlier gradients also retain their influence even when the optimizer moves into a different region of the loss landscape.

AdaGrad demonstrates the benefit of adapting learning rates to individual parameters, but it also reveals the limitations of retaining the entire gradient history. This motivates RMSProp, which replaces the cumulative sum with a moving average that gives greater importance to more recent gradients.

E. RMSProp and Exponential Gradient Statistics

AdaGrad adapts the learning rate of each parameter based on its gradient history. However, its cumulative

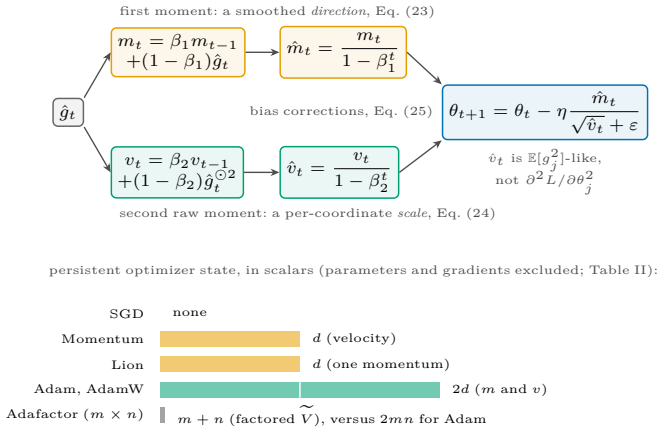


Fig. 5: Adam builds two different gradient statistics. The first moment estimates direction; the second moment controls coordinate-wise scaling. Neither equals the Hessian.

approach has a limitation: the accumulated squared gradients continue to increase, which can cause the effective learning rates to become too small. RMSProp addresses this problem by replacing the cumulative sum with an exponential moving average of squared gradients.

The update is expressed as

$$s_t = \rho s_{t-1} + (1-\rho)\hat{g}_t^{\odot 2}, \qquad \theta_{t+1} = \theta_t - \eta \frac{\hat{g}_t}{\sqrt{s_t + \varepsilon}}. \quad (22)$$

Here, s_t represents the moving average of squared gradients, while ρ controls how much weight is given to previous information. The operations are applied separately to each parameter. Unlike AdaGrad, which retains the cumulative contribution of all previous squared gradients, RMSProp gradually reduces the influence of older observations.

This difference allows RMSProp to respond when gradient magnitudes change during training. If a parameter begins receiving larger gradients, its moving average increases and reduces the effective learning rate. If its gradients become smaller, the moving average can decrease, allowing larger updates again. This makes the method more responsive to changes in the loss landscape.

The choice of ρ determines how quickly this adjustment occurs. A value close to one produces a smoother estimate that changes slowly, while a smaller value gives more importance to recent gradients. The appropriate balance depends on how much the gradients fluctuate during training. RMSProp was popularized through Hinton's lecture material [30], while AdaDelta introduced a related approach based on running gradient statistics [31].

RMSProp therefore improves upon AdaGrad by allowing adaptive learning rates to change throughout training. However, it still uses the current gradient to determine the update direction. The next method, Adam, builds on this idea by combining adaptive scaling with momentum, allowing both the direction and magnitude of updates to depend on gradient history.

F. Adam: Momentum Combined with Adaptive Scaling

Momentum and RMSProp address different limitations of gradient descent. Momentum uses past gradients to reduce fluctuations in the update direction, while RMSProp adjusts the effective learning rate of each parameter according to recent gradient magnitudes. Adam combines these mechanisms to adapt both the direction and scaling of parameter updates [4].

To achieve this, Adam maintains two moving averages of the stochastic gradient:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1)\hat{g}_t, \quad (23)$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2)\hat{g}_t^{\odot 2}. \quad (24)$$

The first estimate, m_t, is a moving average of past gradients and plays a role similar to momentum. The second, v_t, tracks recent squared-gradient magnitudes and determines how strongly each parameter's update is scaled. The coefficients β_1 and β_2 control how much historical information each estimate retains. These two components are illustrated in Fig. 5.

Since both moving averages are initialized at zero, their values are initially biased toward zero. Adam addresses this through bias correction before computing the parameter update:

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1-\beta_2^t},$$

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \varepsilon}}. \quad (25)$$

The corrected first moment provides the update direction, while the corrected second moment controls its magnitude for each parameter. As a result, parameters with different gradient histories can receive differently scaled updates even when they share the same global learning rate.

Although Adam adjusts updates using gradient statistics, these statistics should not be confused with curvature information. In particular, the second moment measures squared-gradient magnitudes rather than the second derivatives represented by the Hessian. Adam is therefore classified as an adaptive first-order method, not a second-order optimizer.

By combining momentum with adaptive scaling, Adam provides a practical approach to handling noisy gradients and differences in parameter sensitivity. However, its additional moving averages require memory, and its adaptive update rule introduces convergence considerations that are not present in ordinary gradient descent. These limitations motivate the modifications discussed in the following subsections.

G. Adam's Convergence Caveat and AMSGrad

Adam combines momentum with adaptive scaling to improve how gradients are used during training. However, this flexibility also makes its convergence behavior more difficult to analyze. In particular, the effective learning

rates can change throughout training as the moving averages of squared gradients increase or decrease. These changes do not always produce stable long-term progress.

Reddi, Kale, and Kumar [32] demonstrated this limitation by constructing an online convex optimization example in which the original Adam algorithm fails to converge. Their analysis showed that the way Adam updates its second-moment estimate can lead to problematic changes in the effective learning rates. This result does not imply that Adam generally fails in practice. Rather, it establishes that its original update rule does not guarantee convergence under all conditions considered in the analysis.

To address this issue, the authors proposed AMSGrad. The method modifies Adam by maintaining a running maximum of its second-moment estimates:

$$\bar{v}_t = \max(\bar{v}_{t-1}, v_t). \tag{26}$$

The maximum is applied separately to each parameter. Unlike Adam's original second-moment estimate, this running maximum cannot decrease. As a result, the corresponding scaling denominator cannot become smaller, preventing one source of increases in the effective learning rate. This modification supports stronger convergence guarantees under suitable assumptions [32].

Subsequent studies have continued to investigate the convergence properties of adaptive optimization methods and the conditions required for stable training [33], [34], [35]. Other variants, including RAdam, AdaBelief, and AdaBound, propose different modifications to Adam's adaptive behavior [36]–[38]. These approaches reflect the broader effort to improve the reliability of adaptive optimization without abandoning its practical advantages.

The distinction between Adam and AMSGrad highlights an important consideration when evaluating optimizers. Strong empirical performance does not necessarily imply a general convergence guarantee, and theoretical improvements do not automatically translate into faster training. Beyond convergence, another design choice also affects Adam's behavior: how regularization is incorporated into its parameter updates. This motivates AdamW, discussed next.

H. AdamW: Why Regularization and Weight Decay Are Different

Adam adapts parameter updates using historical gradient information. While this can improve optimization, it also changes how regularization affects the training process. Regularization is commonly used to discourage excessively large model parameters, often by adding an L2 penalty to the loss function. For a parameter vector θ, this penalty takes the form $\frac{\lambda}{2}\|\theta\|_2^2$, where λ controls its strength.

In standard gradient descent, adding this penalty to the loss produces the update

$$\theta_{t+1} = (1 - \eta\lambda)\theta_t - \eta g_t. \tag{27}$$

The factor $1-\eta\lambda$ reduces the magnitude of the parameters before the gradient update is applied. This is known as *weight decay*. Under ordinary gradient descent, adding an L2 penalty and applying weight decay are mathematically equivalent.

However, this equivalence does not generally hold for adaptive optimizers such as Adam. When the regularization term is included in the gradient, Adam scales it using the same parameter-specific statistics applied to the loss gradient. Consequently, the amount of parameter shrinkage depends on the adaptive scaling of each coordinate. This differs from conventional weight decay, which applies shrinkage independently of the gradient statistics.

AdamW addresses this issue by separating weight decay from the adaptive gradient update. Its update can be expressed as

$$\theta_{t+1} = (1 - \eta\lambda)\theta_t - \eta\frac{\hat{m}_t}{\sqrt{\hat{v}_t + \varepsilon}}. \tag{28}$$

The first term applies weight decay directly to the parameters, while the second performs Adam's adaptive update. By separating these operations, AdamW allows the weight-decay strength to be controlled without being scaled by the adaptive preconditioner. Loshchilov and Hutter [39] introduced this distinction and studied its effect on neural-network training.

AdamW therefore does not change the central mechanism through which Adam estimates gradient direction and scale. Instead, it changes how regularization is applied. This distinction illustrates how optimizer behavior depends not only on the gradient transformation itself but also on the additional operations included in the update. The next subsection considers another practical concern: the memory required to store Adam's gradient statistics in large neural networks.

I. Adafactor: Factoring a Large Second-Moment State

Adam maintains two moving averages for each model parameter. Although these statistics help determine the direction and magnitude of updates, storing them becomes expensive as neural networks grow. For a weight matrix containing $m \times n$ parameters, Adam requires two additional arrays of the same size. Adafactor addresses this memory cost by reducing the storage needed for the second-moment estimate [5].

Instead of maintaining a separate squared-gradient statistic for every entry in a weight matrix, Adafactor stores summaries along its rows and columns. These smaller statistics are then used to approximate the full second-moment matrix. For a matrix with m rows and n columns, this reduces the second-moment storage from mn values to approximately $m + n$.

The underlying idea is that the row and column statistics can capture useful differences in gradient magnitudes without storing every individual value. However, this reduction comes at the cost of approximation. Since the full second-moment matrix is reconstructed from more compact information, some relationships between its entries may not be preserved. Adafactor therefore trades

the precision of its adaptive scaling estimates for lower memory requirements.

The benefit becomes particularly significant for large weight matrices, where the number of stored row and column statistics can be much smaller than the number of parameters. Practical implementations also include mechanisms for controlling update magnitudes and may omit the first-moment estimate to reduce memory further [5]. Consequently, the actual memory savings depend on the chosen configuration and the shapes of the model's parameter tensors.

Adafactor demonstrates that the memory cost of adaptive optimization can be reduced without abandoning parameter-dependent scaling. While it compresses the statistics used to determine update magnitudes, other methods simplify the update rule itself. The next subsection introduces Lion, which uses momentum information to determine the signs of parameter updates rather than scaling them directly by their gradient magnitudes.

J. Lion: Sign-Based Momentum With One Main State

Adafactor reduces optimizer memory by compressing the statistics used for adaptive scaling. Lion takes a different approach by simplifying the update rule itself. Rather than maintaining separate estimates of gradient direction and magnitude, Lion uses momentum information to determine the direction of each parameter update [11].

Lion constructs its update using a combination of the current gradient and previously accumulated momentum. It then applies a sign operation, which determines whether each parameter should increase or decrease. The update can be expressed as

$$c_t = \beta_1 m_{t-1} + (1-\beta_1)\hat{g}_t, \tag{29}$$

$$\theta_{t+1} = (1-\eta\lambda)\theta_t - \eta\ \mathrm{sign}(c_t), \tag{30}$$

$$m_t = \beta_2 m_{t-1} + (1-\beta_2)\hat{g}_t. \tag{31}$$

Here, c_t combines the current gradient with the previous momentum state, while m_t maintains the history used in subsequent updates. The sign operation converts each component of c_t into a direction. As a result, each nonzero component receives an update of magnitude η, before accounting for weight decay.

This differs from Adam, where the update magnitude depends on the relationship between the first- and second-moment estimates. In Lion, gradient magnitudes influence the accumulated direction, but they do not directly determine the size of the final signed update. This allows Lion to operate with one main momentum state rather than Adam's two moving-average arrays.

The simpler state reduces optimizer memory, although it does not necessarily make training faster or more accurate. Lion was developed through symbolic program search, and its original study reported improvements in several training settings [11]. However, its performance depends on the task and the choice of hyperparameters. In particular, the fixed magnitude of signed updates changes how the learning rate and weight decay should be tuned.

Lion therefore illustrates another way to balance update design and computational efficiency. While Adafactor reduces the information stored for adaptive scaling, Lion changes how gradient history is used to construct updates. Both methods demonstrate that optimizer efficiency can be improved through different design choices, without necessarily introducing curvature information.

K. Other Modern First-Order Designs and a Synthesis

The methods discussed so far illustrate how first-order optimization has evolved to address different limitations of gradient descent. This development continues through approaches that focus on other aspects of training. For example, D-Adaptation aims to reduce the need for manual learning-rate selection under specific theoretical settings [40]. Schedule-free optimization seeks to reduce reliance on predefined learning-rate schedules [41], while Lookahead combines fast parameter updates with a more slowly evolving set of weights [42]. These methods demonstrate that optimizer design is not limited to changing gradient directions or introducing adaptive scaling.

Despite their differences, the methods examined in this section share a common foundation. They use gradients to determine parameter updates, but differ in how they incorporate information from previous iterations. Momentum improves the consistency of update directions, while methods such as Adam adjust the scaling of individual parameters. More recent approaches address the memory requirements of these mechanisms or modify how gradient information is converted into updates. Each design introduces its own trade-offs, making its effectiveness dependent on the training problem and computational resources.

An important limitation remains. Although first-order methods can adapt their updates using gradient history, they do not explicitly describe how the gradient changes across parameter directions. As discussed in Section II, this information is captured by the curvature of the loss function. Curvature becomes particularly relevant when the loss landscape contains narrow or rotated valleys, where ordinary gradient scaling may lead to inefficient movement.

This motivates the study of second-order optimization methods. Rather than relying only on the current gradient and its history, these approaches use information about how the loss changes locally to construct parameter updates. Section IV examines how curvature can improve optimization and why incorporating it introduces additional computational challenges.

IV. Second-Order Optimization: What Curvature Adds (RQ2)

A. Deriving Newton's Direction From the Quadratic Model

First-order optimization methods use gradient information to guide parameter updates. While these methods can improve the efficiency of gradient descent, they do

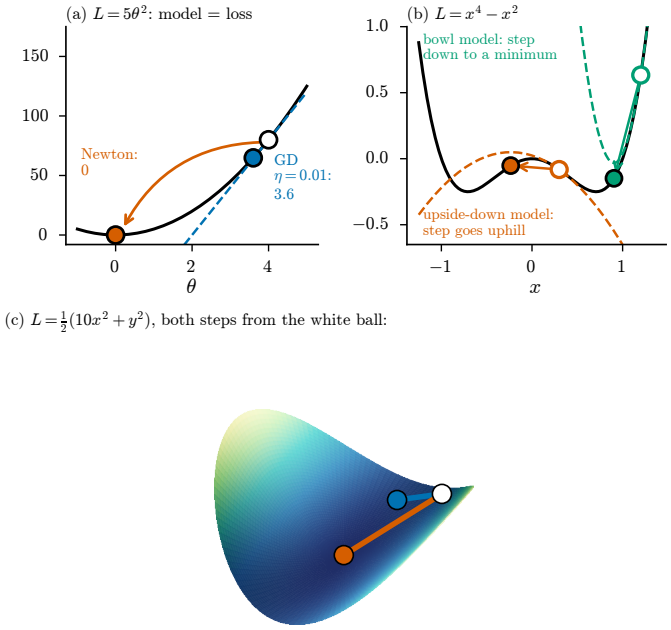


Fig. 6: Newton minimizes a quadratic local model. For an exactly quadratic positive-definite loss, the full step reaches the optimum; for a changing non-convex loss, the quadratic model may be inaccurate or unbounded below.

not explicitly account for how the gradient changes as the parameters move. Newton's method addresses this limitation by incorporating the Hessian, which describes the local curvature of the loss function.

As introduced in Section II, the loss near the current parameters can be approximated by a quadratic function. Unlike the linear approximation used in gradient descent, this model includes a curvature term that describes how the slope changes along different directions. Newton's method uses this quadratic model to determine an update that moves toward its minimum.

When the Hessian H_t is positive definite, minimizing the quadratic model leads to the Newton equation

$$H_t \Delta_t = -g_t, \qquad \theta_{t+1} = \theta_t + \Delta_t. \tag{32}$$

Here, g_t is the current gradient and Δ_t is the proposed parameter update. The Hessian determines how the gradient is adjusted according to the local curvature. This differs from standard gradient descent, which scales all gradient components using a single learning rate.

The advantage can be understood by considering a loss function with different curvature across its parameter directions. Gradient descent may require a small learning rate to remain stable in steep directions, which slows progress along flatter ones. Newton's method accounts for these differences when constructing its update. It generally reduces movement along directions with high positive curvature while allowing larger corrections along directions with lower positive curvature.

For an exactly quadratic loss with a positive-definite Hessian, a full Newton step reaches the minimum in a single iteration. Figure 6 illustrates this behavior and compares it with ordinary gradient descent. However, deep neural-network losses are not generally quadratic. The Hessian also changes as the parameters move, so the local approximation may become inaccurate over larger steps.

Newton's method therefore provides a more informed update by using curvature, but its effectiveness depends on how accurately the quadratic model represents the loss near the current parameters. Computing and using Hessian information can also be expensive for large neural networks. These considerations motivate the analysis of Newton's convergence properties and the practical modifications discussed in the following subsections.

B. Local Quadratic Convergence and Its Assumptions

Newton's method can reach the minimum of a positive-definite quadratic function in a single step because its local quadratic model matches the true objective. For more general loss functions, this exact behavior is no longer guaranteed. However, Newton's method can still converge rapidly when the parameters are sufficiently close to a suitable local minimum.

This behavior is known as *local quadratic convergence*. It means that the distance between the current parameters and the minimum can decrease at a rate proportional to the square of the previous distance. As the parameters approach the minimum, the error can therefore shrink increasingly quickly. This is a stronger form of local convergence than the linear convergence typically associated with basic gradient descent under suitable conditions.

The guarantee depends on several assumptions. The Hessian must be positive definite at the minimum and vary smoothly in its neighborhood. The initial parameters must also be sufficiently close to that minimum. Under these conditions, the quadratic model becomes increasingly accurate as optimization progresses, allowing Newton's updates to make rapid local progress [18], [1].

These assumptions are particularly important in deep learning. Neural-network loss functions are non-convex, and training usually begins far from a known minimum. Consequently, the Hessian may not satisfy the required conditions throughout optimization. Even when Newton's method converges in fewer iterations, the cost of computing curvature information and determining each update may outweigh the reduction in iteration count.

Local quadratic convergence therefore explains the theoretical appeal of Newton's method, but it does not imply that Newton is always the most efficient choice for training neural networks. Its practical effectiveness depends on both the local geometry of the loss and the cost of using curvature information. The next subsection examines a more fundamental difficulty: what happens when the Hessian contains negative curvature and the quadratic model no longer describes a local minimum.

C. Why Undamped Newton Can Fail in Non-Convex Problems

Newton's method uses the Hessian to construct parameter updates based on a local quadratic approximation of the loss. As discussed in the previous subsection, this approach can be highly effective near a minimum where the Hessian is positive definite. However, neural-network loss functions are non-convex, meaning that the Hessian may also contain negative eigenvalues. In such regions, the quadratic approximation does not necessarily describe a local minimum.

Consider the function $L(x) = x^4 - x^2$. At $x = 0$, the gradient is zero, but the second derivative is negative. This point is a local maximum rather than a minimum. More generally, when the Hessian has negative eigenvalues, the quadratic model may curve downward along certain directions. As a result, the Newton update is not guaranteed to point toward decreasing loss. This limitation is illustrated in Fig. 6(b), where the local quadratic model can produce an unsuitable update.

One way to address this problem is through *damping*. Instead of using the Hessian directly, damped Newton methods add a positive term to its diagonal:

$$(H_t + \lambda_t I)\Delta_t = -g_t. \tag{33}$$

Here, λ_t is a damping parameter and I is the identity matrix. Increasing λ_t shifts the Hessian eigenvalues upward. If the modified matrix becomes positive definite, the resulting update is a descent direction whenever the gradient is nonzero.

Damping therefore makes Newton's method more reliable in regions where the original Hessian is unsuitable. However, the choice of λ_t matters. Too little damping may leave problematic curvature unchanged, while excessive damping reduces the influence of the Hessian and makes the update behave more like a scaled gradient descent step.

Even with damping, the quadratic model may be inaccurate when the proposed update is large. Additional safeguards can therefore be used to control how far the optimizer moves or to verify that an update actually reduces the loss. The next subsection introduces trust-region methods and cubic regularization, which address this issue by limiting how strongly the optimizer relies on its local quadratic approximation.

D. Trust Regions and Cubic Regularization

Damping improves the reliability of Newton's method by modifying the Hessian when its curvature information is unsuitable. However, it does not fully address another limitation of the quadratic approximation. Since this model describes the loss locally, its predictions may become inaccurate when the optimizer takes a large step. Trust-region methods address this problem by restricting how far the parameters can move during each update.

A trust-region method defines a region around the current parameters within which the quadratic model is considered sufficiently reliable. The optimizer then searches for an update that reduces the model's predicted loss while remaining inside this region. After applying the update, the method compares the actual reduction in loss with the reduction predicted by the model.

This comparison determines how much the optimizer should trust its approximation in subsequent iterations. If the predicted and actual reductions agree closely, the trust region may be expanded to allow larger updates. If they differ significantly, the region can be reduced to encourage more conservative steps. In this way, the optimizer adapts its update size according to the observed accuracy of its local model [1], [43].

Cubic regularization provides another approach to controlling Newton-style updates. Instead of imposing a fixed restriction on the step length, it adds a penalty that grows with the cube of the step magnitude. This discourages excessively large updates, particularly when the quadratic model predicts unrealistic improvements. The additional penalty also helps ensure that the local optimization problem remains well-defined when the Hessian contains negative curvature.

Both approaches aim to make curvature-based optimization more reliable, but they control the update differently. Trust-region methods impose an explicit limit on the step length, while cubic regularization discourages large steps through an additional penalty. Their theoretical properties have been studied extensively, including convergence guarantees under suitable smoothness assumptions [44], [45].

These methods demonstrate that using curvature information is not sufficient on its own. The optimizer must also determine how much confidence to place in the local model. However, both approaches can require expensive curvature calculations and additional work to determine each update. This motivates methods that approximate curvature using information already collected during optimization, beginning with quasi-Newton methods.

E. BFGS: Learning Curvature From Differences in Gradients

Newton's method uses the Hessian to account for how the gradient changes across parameter directions. While this information can improve optimization, computing the full Hessian is often impractical for large neural networks. Quasi-Newton methods address this limitation by constructing an approximation of the Hessian from information collected during previous updates.

One of the best-known quasi-Newton methods is BFGS, named after Broyden, Fletcher, Goldfarb, and Shanno [46]–[49]. Its central idea is that changes in gradients provide information about the curvature of the loss function. If the gradient changes significantly after a small parameter update, this suggests that the loss has substantial curvature along that direction. Smaller changes indicate lower curvature.

BFGS uses this relationship by recording how the parameters and gradients change between successive iterations:

$$s_t = \theta_{t+1} - \theta_t, \qquad y_t = g_{t+1} - g_t. \tag{34}$$

Here, s_t represents the parameter displacement, while y_t represents the corresponding change in the gradient. The method uses these quantities to update its approximation of the Hessian. The approximation is constructed so that it remains consistent with the observed relationship between the latest parameter displacement and gradient change.

As optimization progresses, BFGS refines its curvature estimate using the information obtained from previous steps. Under appropriate conditions, the approximation remains positive definite, allowing the method to construct descent directions without explicitly evaluating second derivatives. This makes BFGS an alternative to Newton's method when direct Hessian computation is too expensive.

However, estimating curvature from past updates introduces its own challenges. The approximation must be stored and updated, which can become expensive for models with many parameters. Its quality also depends on how reliably gradient changes reflect the underlying curvature. In stochastic training, minibatch noise can make these differences less informative.

BFGS therefore reduces the need for direct Hessian computation, but it does not completely eliminate the memory and computational challenges of curvature-aware optimization. This motivates limited-memory variants, which retain only a small amount of information from recent updates rather than maintaining a full curvature approximation.

F. Inverse BFGS and Limited-Memory BFGS

BFGS avoids computing the Hessian directly by estimating curvature from changes in parameters and gradients. However, the resulting approximation is a full matrix, which can become expensive to store when the model contains many parameters. For a model with d parameters, this matrix requires $O(d^2)$ memory, making standard BFGS difficult to apply to large neural networks.

One variation of BFGS maintains an approximation of the inverse Hessian rather than the Hessian itself. This allows the optimizer to compute the update direction by applying the inverse approximation directly to the gradient. Although this simplifies how the update is calculated, it does not solve the memory problem because the inverse approximation is also a full matrix.

Limited-memory BFGS (L-BFGS) addresses this limitation by avoiding the storage of either matrix. Instead, it retains a small number of recent parameter displacements and their corresponding gradient changes, as defined in Eq. (34). These observations contain information about how the loss changes along the directions explored during optimization. L-BFGS uses them to approximate the action of the inverse Hessian when constructing an update.

If the method stores k pairs of parameter and gradient differences, its memory requirement becomes $O(kd)$ rather than $O(d^2)$. When k is much smaller than the number of parameters, this represents a substantial reduction. The method is therefore able to incorporate curvature information without maintaining a full matrix approximation [50], [51].

However, lower memory requirements do not automatically make L-BFGS more efficient than first-order methods. Its updates may require additional gradient evaluations or line searches. Moreover, the gradient differences used to estimate curvature can become unreliable when they are affected by minibatch noise. These issues make the method particularly sensitive to how curvature information is collected during training.

L-BFGS demonstrates that useful curvature information can be extracted from a limited history of optimization steps. However, it still relies on past observations to construct an approximation. Another approach is to obtain curvature information directly without storing the Hessian. This motivates Hessian-free optimization, discussed next.

Algorithm 1 One outer Hessian-free step (schematic)

1: Evaluate gradient g_t at θ_t.
2: Define operator $A_t v = G_t v + \lambda_t v$ through matrix–vector products.
3: Approximately solve $A_t \Delta = -g_t$ using CG.
4: Accept, damp, or rescale Δ using model-quality safeguards.
5: Update $\theta_{t+1} = \theta_t + \Delta$.

G. Hessian–Vector Products and Hessian-Free Optimization

L-BFGS reduces the memory cost of curvature-aware optimization by using a limited history of gradient changes. However, these observations provide only an approximation of the curvature encountered during previous updates. Hessian-free optimization offers another approach by accessing curvature information directly without constructing or storing the full Hessian.

The main idea is to compute a *Hessian–vector product*, written as Hv, where H is the Hessian and v is a chosen direction. This product describes how the gradient changes locally along that direction. Unlike explicitly computing the Hessian, a Hessian–vector product can be evaluated efficiently using automatic differentiation [52], [53].

This makes it possible to construct a Newton-like update without forming the Hessian matrix. Recall from Eq. (32) that Newton's method requires solving a system involving the Hessian and the current gradient. Hessian-free methods approach this problem using iterative solvers, such as conjugate gradients, which can progressively approximate the update using Hessian–vector products.

Rather than solving the entire system through a direct matrix factorization, the iterative solver improves its estimate over several internal steps. Each step uses curvature

information along selected directions. The method therefore avoids the memory cost of a full Hessian, although determining a single parameter update may require multiple curvature-product evaluations.

In practice, Hessian-free methods often use damping to improve stability. They may also replace the Hessian with a more suitable curvature approximation, such as the generalized Gauss–Newton matrix. This can help avoid difficulties caused by negative curvature in non-convex loss functions. Martens [54] explored Hessian-free optimization for deep learning, and subsequent work extended the approach to recurrent neural networks [55].

Hessian-free optimization demonstrates that curvature information can be useful even when the full Hessian is too large to store. However, avoiding matrix storage does not eliminate the computational cost of solving for an update. Its efficiency depends on the number of internal solver iterations and the cost of evaluating curvature products.

The discussion so far has introduced several ways to use or approximate curvature. However, not every matrix used in an optimization method represents the Hessian. The next subsection distinguishes the Hessian from two other important matrices: the Gauss–Newton matrix and the Fisher information matrix.

H. Hessian, Gauss–Newton, and Fisher: Three Different Matrices

The methods discussed so far use curvature information to improve parameter updates. However, the matrices used in curvature-aware optimization do not always describe the same mathematical properties. In particular, the Hessian, generalized Gauss–Newton matrix, and Fisher information matrix have different interpretations. Understanding these differences is important because their use can lead to different optimization behavior.

The **Hessian**, defined as $H = \nabla^2 L(\theta)$, contains the second derivatives of the loss function with respect to the model parameters. It describes how the gradient changes locally and therefore captures the curvature of the training objective. As discussed earlier, the Hessian can have negative eigenvalues in non-convex problems. This can make Newton-style updates unstable or unsuitable for reducing the loss.

The **generalized Gauss–Newton (GGN) matrix** provides an alternative way to represent curvature. Rather than using all the second-order information contained in the Hessian, it constructs an approximation based on how changes in the model parameters affect its outputs and, consequently, the loss. For common losses that are convex with respect to the model outputs, the resulting matrix is positive semidefinite. This avoids the negative-curvature difficulty associated with the full Hessian and can make curvature-based updates easier to compute and control [53], [56].

The **Fisher information matrix** has a different interpretation. It arises from probabilistic models and measures how sensitive the model's predicted probability distribution is to changes in its parameters. While the Hessian describes the curvature of a chosen loss function, the Fisher matrix describes changes in the statistical behavior of the model. This makes it particularly relevant to natural-gradient methods, which seek parameter updates that account for how strongly those changes affect the model's predictions.

Although these matrices may be closely related under specific assumptions, they are not generally interchangeable. The distinction also applies to the empirical Fisher, which is constructed from gradients evaluated on observed data and does not necessarily equal the Fisher information matrix. Treating all these quantities as equivalent can therefore lead to misleading interpretations of optimization methods [56], [57], [58].

The central distinction is between curvature of the training objective and sensitivity of the model's predictions. Both can provide useful information for optimization, but they describe different aspects of the problem. This motivates natural-gradient optimization, which uses the Fisher information matrix to determine parameter updates based on changes in the model's predicted distributions rather than relying only on distances between parameter values.

I. Natural Gradient: Geometry of Predictions Rather Than Coordinates

The previous subsection introduced the Fisher information matrix as a measure of how sensitive a model's predictions are to changes in its parameters. Natural-gradient optimization uses this information to construct parameter updates that account for the model's statistical behavior rather than relying only on the numerical values of its parameters.

In standard gradient descent, the size of an update is measured in parameter space. However, equal changes in different parameters do not necessarily have equal effects on the model's predictions. A small change in one parameter may substantially alter the predicted probability distribution, while a much larger change in another may have little effect. Natural gradient addresses this difference by considering how parameter updates affect the model's output distribution.

The Fisher information matrix provides a way to measure this sensitivity. Natural gradient uses it to adjust the ordinary gradient, giving the update

$$\Delta_t = -\eta F(\theta_t)^{-1} g_t. \tag{35}$$

Here, $F(\theta_t)$ is the Fisher information matrix and g_t is the gradient of the loss. The inverse Fisher matrix transforms the gradient according to how strongly different parameter directions affect the model's predicted distribution. Directions associated with greater statistical sensitivity are generally scaled more conservatively.

This approach differs from Newton's method. Although both methods use a matrix to transform the gradient, Newton relies on the curvature of the loss function, while natural gradient uses the geometry of the model's probability distributions. An important consequence is that

natural gradient is designed to behave consistently under smooth changes in parameter coordinates, provided the relevant mathematical assumptions hold [57], [59], [60].

The main challenge is computational. For large neural networks, the Fisher matrix can be too expensive to construct and invert directly. Practical implementations therefore require approximations and numerical safeguards. This motivates methods such as K-FAC, which exploit the structure of neural-network layers to approximate Fisher information more efficiently.

J. What the Theory Really Guarantees

The methods discussed in this section demonstrate how curvature information can improve optimization. However, their theoretical advantages depend on the assumptions under which they are analyzed. Understanding these assumptions is essential when comparing their convergence properties with those of first-order methods.

For example, Newton's method can achieve local quadratic convergence when the objective is sufficiently smooth and the parameters are close to a suitable minimum [1]. Under appropriate conditions, BFGS can also achieve superlinear convergence, meaning that its progress accelerates as it approaches the solution [51]. These results demonstrate the potential benefits of using curvature, but they do not describe the behavior of these methods from arbitrary starting points in a non-convex neural-network loss landscape.

Other approaches provide guarantees that apply more broadly to non-convex optimization. Trust-region methods and cubic regularization can establish convergence toward suitable stationary points under specific assumptions [44], [45]. However, these guarantees describe mathematical progress rather than the total computational effort required to train a model. Each iteration may involve additional curvature calculations or an expensive internal optimization procedure.

Consequently, a method with a stronger theoretical convergence rate is not necessarily faster in practice. Its performance also depends on the cost of each update and how accurately its curvature information represents the loss. Moreover, reaching a low training loss does not guarantee better generalization to unseen data.

The main advantage of second-order optimization is therefore its ability to use information about the local behavior of the loss that ordinary gradient descent does not explicitly model. The challenge is obtaining and applying this information efficiently. This motivates the structured optimization methods examined in Section V, which seek to capture useful geometric information while reducing the computational and memory costs of full-matrix approaches.

V. Structured Curvature and Modern Matrix-Aware Methods (RQ2)

A. Why Structure Matters in Neural Networks

The previous section showed that curvature-aware methods can improve optimization by accounting for relationships between parameter directions. However, representing these relationships through full matrices becomes increasingly expensive as neural networks grow. Structured optimization methods address this difficulty by exploiting the organization of parameters within neural-network layers.

Consider a fully connected layer with a weight matrix $W \in \mathbb{R}^{m\times n}$. This matrix contains mn parameters, but its entries are not simply an unrelated collection of values. Each row contributes to an output feature, while each column corresponds to an input feature. This organization provides information that can be useful when constructing parameter updates.

A conventional coordinate-wise optimizer treats each matrix entry independently when determining its scaling. While this is computationally efficient, it does not explicitly capture relationships between rows or columns. A full-matrix approach can represent interactions between all parameters, but would require a matrix containing $(mn)^2$ entries for this single layer. For large weight matrices, this is generally impractical.

Structured methods seek a balance between these approaches. Rather than modeling every parameter interaction separately, they use smaller matrices or approximations that preserve useful aspects of the layer's structure. This can provide more information than independent parameter scaling while reducing the memory and computational demands of full-matrix methods.

However, exploiting matrix structure does not necessarily mean estimating the Hessian. Different methods use this structure for different purposes. K-FAC approximates Fisher information using smaller matrices associated with a layer. Shampoo uses gradient statistics to adapt updates across matrix dimensions. Muon takes another approach by transforming the matrix update itself through orthogonalization. Although these methods operate on matrix-shaped parameters, they rely on different mathematical principles [61], [6], [7].

The following subsections examine these approaches and the trade-offs involved in their design. We begin with K-FAC, which builds on the natural-gradient framework introduced in Section IV by approximating the Fisher information matrix more efficiently.

B. K-FAC: Approximating Fisher Blocks With Kronecker Products

Natural-gradient optimization uses the Fisher information matrix to account for how changes in model parameters affect the predicted probability distribution. However, applying this approach to large neural networks is difficult because the Fisher matrix grows rapidly with the number of parameters. Kronecker-Factored Approximate Curvature (K-FAC) addresses this limitation by constructing a more manageable approximation of the Fisher matrix [61].

K-FAC takes advantage of the structure of individual neural-network layers. For a fully connected layer, the gradient of its weight matrix depends on two quantities:

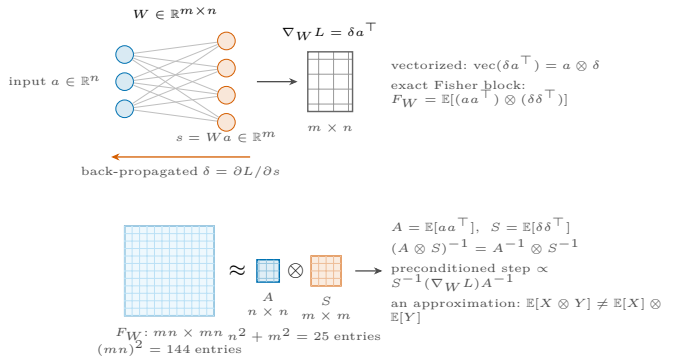


Fig. 7: How K-FAC obtains tractable layer-wise curvature from activation and backpropagated-error statistics. The Kronecker separation is a modeling approximation, not an exact identity for arbitrary data.

the activations received from the previous layer and the error signals propagated backward during training. Both contain information about how the layer contributes to the loss. K-FAC uses statistical relationships within these quantities to approximate the layer's Fisher information.

Rather than representing every interaction between the layer's parameters, K-FAC approximates the corresponding Fisher block using the Kronecker product of two smaller matrices:

$$F_W \approx A \otimes S. \tag{36}$$

Here, A represents statistics of the layer's input activations, while S represents statistics of the backpropagated error signals. The symbol $\otimes$ denotes the Kronecker product, which combines the two matrices to represent a larger structured matrix. This approximation is illustrated in Fig. 7.

The main advantage of this factorization is that K-FAC can work with the two smaller matrices instead of directly constructing and inverting a large Fisher block. The resulting information is then used to transform the layer's gradient, producing a natural-gradient-style update at a lower computational and memory cost.

However, the factorization is an approximation rather than an exact representation of the Fisher matrix. Its accuracy depends on how well the smaller matrices capture the statistical relationships within the layer. K-FAC also introduces computational overhead because these matrices must be estimated and periodically processed. Its practical efficiency therefore depends on the network architecture and the frequency of these calculations. The approach has also been extended beyond fully connected layers, including convolutional networks [62].

K-FAC demonstrates how preserving layer structure can make information-geometric optimization more practical. However, it remains closely connected to the Fisher information matrix. Other structured methods use different information to construct their updates. Shampoo, discussed next, uses accumulated gradient statistics to adapt parameter updates across matrix dimensions rather than approximating Fisher curvature.

C. Shampoo: Preconditioning Across Tensor Dimensions

K-FAC demonstrates how the structure of neural-network layers can make natural-gradient optimization more practical. However, exploiting this structure does not necessarily require estimating Fisher information. Shampoo takes a different approach by using accumulated gradient statistics to determine how parameter updates should be scaled [6].

Recall that adaptive methods such as Adam adjust each parameter independently based on its gradient history. While this helps account for differences in gradient magnitudes, it does not explicitly capture relationships between parameters. This limitation becomes important for weight matrices, where changes along different rows and columns may be related. Shampoo addresses this by adapting updates across these matrix dimensions rather than treating every parameter separately.

For a weight matrix W, Shampoo maintains two matrices that summarize its gradient history. One captures relationships along the rows, while the other captures relationships along the columns. These statistics are then used to transform the current gradient, allowing the optimizer to account for directions that involve multiple parameters.

For a matrix-shaped gradient G_t, a representative Shampoo update is

$$\Delta W_t = -\eta L_t^{-1/4} G_t R_t^{-1/4}, \tag{37}$$

where L_t and R_t are the accumulated row and column gradient statistics. The matrix transformations on either side of G_t adjust the update according to these statistics. Unlike Adam's coordinate-wise scaling, this approach can account for relationships between different parameter directions.

This additional information comes at a computational cost. Although Shampoo avoids constructing a full curvature matrix, it must store and process its row and column statistics. For a weight matrix of size $m \times n$, these matrices require approximately $m^2 + n^2$ stored values. Computing their inverse matrix roots also introduces additional work, which can become expensive for large layers. Practical implementations therefore use techniques that reduce how frequently these operations are performed [63].

Shampoo's original theoretical analysis provides convergence guarantees under stochastic convex optimization assumptions [6]. These results do not automatically extend to general non-convex neural-network training. Its practical effectiveness must also account for the additional cost of maintaining and applying the matrix transformations.

Shampoo illustrates how structured gradient statistics can provide more informative updates than independent parameter scaling. However, it does not directly estimate the Hessian or Fisher information matrix. The next subsection returns to curvature-aware optimization and considers methods that use simpler diagonal approximations to reduce computational cost.

D. Curvature-Aware Diagonal Alternatives: AdaHessian and Apollo

Shampoo improves upon coordinate-wise adaptation by using matrix statistics that capture relationships between parameter directions. However, the additional information comes at a computational cost. Another approach is to retain the efficiency of diagonal scaling while incorporating information about the curvature of the loss function.

AdaHessian follows this approach by estimating the diagonal of the Hessian rather than constructing the complete matrix [64]. Each diagonal entry describes how the gradient of a parameter changes with respect to that same parameter. AdaHessian uses stochastic estimates of these values and smooths them over time to guide parameter updates. This allows it to incorporate second-order information without the memory requirements of a full Hessian.

Apollo also uses a diagonal approximation, but obtains its curvature information differently. Rather than estimating second derivatives directly, it uses changes in parameters and gradients to update a diagonal quasi-Newton approximation [65]. This connects Apollo to the BFGS family discussed in Section IV, while avoiding the cost of maintaining a dense curvature matrix.

Although both methods use diagonal curvature information, they differ in how that information is obtained. AdaHessian estimates second-order derivatives, while Apollo learns an approximation from the behavior of successive updates. Their convergence properties and computational costs therefore depend on different assumptions.

The main limitation of both approaches is that diagonal approximations do not capture interactions between different parameters. As illustrated by the rotated valley in Section II-D, these interactions can be important when the main curvature directions do not align with the parameter coordinates. Nevertheless, diagonal methods may be preferable when the cost of structured matrix operations outweighs their benefits.

AdaHessian and Apollo therefore illustrate a trade-off between the amount of curvature information used and the cost of obtaining it. The next subsection introduces Sophia, which also uses diagonal Hessian information but combines it with a mechanism that explicitly limits the magnitude of parameter updates.

E. Sophia: Diagonal Hessian Estimation With Clipped Updates

AdaHessian and Apollo demonstrate how diagonal curvature approximations can make second-order information more affordable. Sophia follows a related approach, with a particular focus on improving the efficiency of large language model training. It combines momentum with an estimate of the Hessian diagonal to adjust parameter updates according to the local curvature of the loss [12].

The motivation is similar to that of Newton's method. When the loss has high curvature along a parameter direction, a large update may cause instability. In flatter directions, a larger update may allow faster progress. Sophia uses diagonal Hessian estimates to account for these differences without constructing the full Hessian.

Unlike methods that estimate curvature at every iteration, Sophia updates its curvature information periodically. This reduces the overhead of obtaining second-order information. The optimizer then uses the estimates to scale its momentum-based updates, allowing the step size to respond to local curvature.

However, curvature estimates can be noisy or unreliable, particularly during stochastic training. Small estimated curvature values may also produce excessively large updates. To address this, Sophia applies *clipping*, which limits the magnitude of each scaled update. This prevents individual parameter updates from exceeding a specified threshold and provides an additional mechanism for controlling optimization stability.

Sophia therefore combines two complementary ideas. Curvature estimates determine how the updates should be scaled, while clipping limits how large those updates can become. The resulting method aims to benefit from second-order information without the full computational cost of Newton-style optimization.

In the original study, Sophia was evaluated on GPT-style language models ranging from 125 million to 1.5 billion parameters. The authors reported approximately a twofold reduction in the resources required to reach a matched target compared with their Adam baseline [12]. This result applies to the experimental settings considered in that study and should not be interpreted as a universal performance guarantee.

Sophia illustrates how approximate curvature information can be combined with practical safeguards to improve large-scale optimization. However, it still treats parameter directions independently through diagonal estimates. The next subsection introduces SOAP, which instead uses matrix structure to perform adaptive optimization in a transformed coordinate system.

F. SOAP: Adam-Style Adaptation in a Rotated Matrix Basis

The previous subsections showed two approaches to improving parameter updates. Sophia uses diagonal curvature estimates, while Shampoo uses structured gradient statistics to account for relationships between matrix dimensions. SOAP builds on the second approach by combining Shampoo's matrix-based transformations with the adaptive update mechanism of Adam [13].

As discussed in Section II-D, the main directions of a loss landscape do not always align with the individual parameter coordinates. This can limit the effectiveness of methods that scale each parameter independently. Shampoo partially addresses this difficulty by using matrix statistics to identify important directions within the weight matrix. SOAP takes advantage of these directions by performing Adam-style adaptation in a rotated coordinate system.

To achieve this, SOAP uses the eigenvectors of Shampoo's row and column statistics to transform the gradient. If U and V represent the corresponding eigenvector matrices, the transformed gradient is

$$\widetilde{G}_t = U^\top G_t V. \tag{38}$$

Here, G_t is the gradient of the weight matrix, while $\widetilde{G}_t$ represents the same gradient in the transformed coordinates. SOAP then applies adaptive moment estimation in this coordinate system before transforming the resulting update back to the original parameter space.

The motivation is that adaptive scaling may be more effective when applied along directions that better reflect relationships within the weight matrix. Rather than treating each parameter independently in its original coordinates, SOAP allows the adaptation to operate in directions identified through structured gradient statistics. This provides a connection between the coordinate-wise adaptation of Adam and the matrix-based preconditioning of Shampoo.

However, maintaining the transformed coordinate system introduces additional computational work. The matrices used to define it must be estimated and updated during training. SOAP therefore faces a trade-off between the potential benefit of more informative updates and the cost of maintaining the required matrix transformations.

In the original study, the authors reported more than 40% fewer optimization iterations and over 35% lower wall-clock time than AdamW in the evaluated large-batch language-model settings [13]. These results suggest that the additional matrix operations can be worthwhile under suitable conditions. However, the reported improvements depend on the experimental setup and should not be interpreted as universal performance guarantees.

SOAP illustrates how adaptive optimization can benefit from a more suitable coordinate system without directly estimating the Hessian. The next subsection examines Muon, which also transforms matrix-shaped updates but follows a different principle. Rather than applying adaptive scaling in a rotated basis, Muon modifies the geometry of the update matrix through orthogonalization.

G. Muon: Orthogonalizing a Matrix Update Is Not Newton's Method

The methods discussed so far use matrix structure to improve how gradients are scaled. Muon takes a different approach by modifying the matrix update itself. Instead of estimating the Hessian or Fisher information, Muon uses momentum to construct an update and then applies an orthogonalization procedure before adjusting the model parameters [7].

The motivation comes from treating a weight matrix as a matrix rather than a collection of independent parameters. In conventional gradient descent, the magnitude of each gradient component directly influences its update. Muon instead considers how the proposed update acts across different directions of the weight matrix. Its orthogonalization step changes the relative strength of these directions while preserving their underlying orientation.

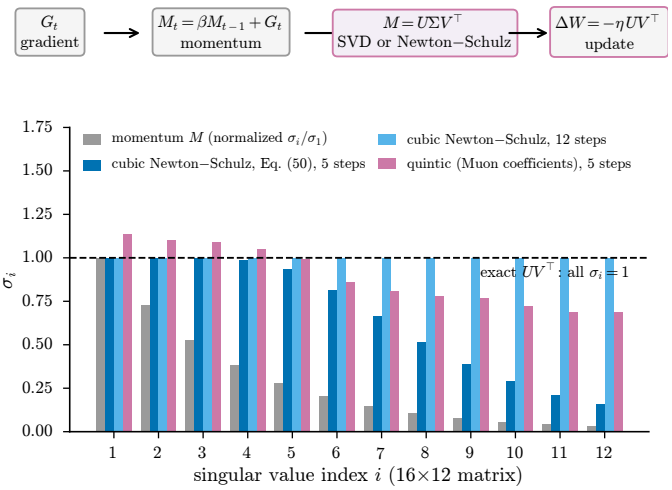


Fig. 8: Matrix geometry in Muon. Orthogonalization changes the singular spectrum of the proposed weight update without estimating the model's Hessian; Newton–Schulz iterations are a numerical device for the matrix operation.

This transformation can be understood through the singular value decomposition (SVD). A momentum matrix M can be decomposed as

$$M = U\Sigma V^\top, \tag{39}$$

where U and V describe the matrix's singular directions, while Σ contains their corresponding singular values. Muon's idealized orthogonalization replaces the nonzero singular values with one, producing

$$\mathrm{msign}(M) = UV^\top. \tag{40}$$

This operation preserves the singular directions but changes how strongly the update acts along them. Unlike Adam, which rescales individual parameters using historical gradient statistics, Muon modifies the update according to its matrix geometry. Figure 8 illustrates this effect by showing how orthogonalization changes the singular values of a momentum matrix.

Computing a full SVD at every iteration would introduce substantial computational overhead. Muon therefore uses Newton–Schulz iterations to approximate the orthogonalization through repeated matrix multiplications. Despite the name, this procedure should not be confused with Newton's optimization method. Newton–Schulz is used here as a numerical technique for transforming a matrix, not for estimating or inverting the Hessian of the training loss.

Muon is primarily designed for suitable matrix-shaped parameters, such as hidden-layer weight matrices. Other parameters may require a different optimizer, and practical implementations often combine Muon with AdamW for this purpose [7]. Its efficiency therefore depends on both the cost of orthogonalization and how the optimizer is applied across the model.

Muon demonstrates that matrix structure can improve optimization through mechanisms other than curvature estimation or adaptive scaling. This distinction is important when comparing modern optimizers, since similar-looking matrix operations may represent different mathematical ideas. The next subsection brings these approaches together by examining what each method estimates or transforms when constructing its updates.

H. What Is Being Inverted, Estimated, or Normalized?

The methods discussed in this section demonstrate that matrix-based optimization can take several forms. Although K-FAC, Shampoo, SOAP, Sophia, and Muon all modify how gradients are used, they rely on different mathematical information. Understanding these differences is important when evaluating what each method contributes to neural-network optimization.

K-FAC uses a structured approximation of the Fisher information matrix to account for how parameter changes affect the model's predictions. Shampoo instead constructs matrices from accumulated gradient statistics to adapt updates across the dimensions of a weight matrix. SOAP builds on these statistics by applying Adam-style adaptation in a transformed coordinate system. Sophia follows a different approach by estimating diagonal Hessian information and limiting the resulting update magnitudes through clipping. Muon does not estimate curvature; it transforms a momentum-based update through matrix orthogonalization [61], [6], [7], [12], [13].

These approaches also differ in how they modify the gradient. *Preconditioning* uses a matrix or related transformation to rescale update directions according to the information available to the optimizer. *Normalization*, in contrast, controls or replaces aspects of the update magnitude according to a chosen mathematical rule. Muon's orthogonalization is an example of a matrix-based transformation that changes the singular values of the update rather than applying conventional curvature-based scaling [66].

These distinctions have practical consequences. Methods that maintain structured matrices can capture relationships between parameter directions, but they introduce additional storage and computational costs. Methods that use diagonal information are generally less expensive, although they cannot fully represent interactions between parameters. Other approaches reduce persistent optimizer state but require additional matrix operations during each update.

The choice between these methods therefore involves more than selecting the most sophisticated mathematical transformation. Their usefulness depends on whether the information they capture leads to meaningful improvements relative to the resources they consume. This is particularly important for large neural networks, where additional optimizer calculations may significantly affect training time.

The methods examined in Sections III–V provide different solutions to the same underlying problem: how to construct effective parameter updates from the information available during training. Having established their mathematical principles, the next section compares their practical advantages and limitations under a common evaluation framework.

VI. Critical Comparison: Theory, Geometry, and Cost

A. Comparing the Mathematics Rather Than the Names

The previous sections examined how different optimization methods address the limitations of gradient descent. Although they share the goal of minimizing the training loss, they differ in the information they use to construct parameter updates. These differences provide a more meaningful basis for comparison than simply grouping methods by their names or the complexity of their update rules.

First-order methods primarily improve how gradient information is used. Momentum incorporates previous gradients to produce more consistent update directions, while Adam adapts the scaling of individual parameters based on their gradient history. These approaches can improve training efficiency without explicitly estimating the curvature of the loss function. However, their effectiveness depends on the behavior of the gradients and the chosen hyperparameters [3], [4].

Curvature-aware methods take a different approach by using information about how the gradient changes as the parameters move. Newton's method obtains this information from the Hessian, while BFGS approximates it using changes observed during optimization. This can produce updates that better account for the local shape of the loss landscape, although obtaining and applying curvature information introduces additional computational costs [1].

Modern structured methods seek to make more informative updates practical for large neural networks. As discussed in Section V, some methods approximate curvature using smaller matrices, while others rely on structured gradient statistics. Muon follows a different principle by modifying the geometry of the update matrix itself. These distinctions matter because similar matrix operations do not necessarily represent the same mathematical information [12], [7].

The comparison therefore depends on understanding what each optimizer attempts to improve and what it requires to do so. A method may produce more effective updates but require greater memory or additional computation. Another may use less information yet offer better efficiency under a limited training budget. The following subsections examine these trade-offs more closely, beginning with the memory required to maintain optimizer information.

B. Optimizer-State Memory: Count the Actual Tensors

Optimization methods differ not only in how they construct parameter updates, but also in how much information they retain between iterations. This information is

known as the *optimizer state*. For large neural networks, storing these additional values can require substantial memory and may limit the size of models that can be trained on available hardware.

The memory requirement depends on the optimizer's design. SGD requires little additional persistent state beyond the model parameters and gradients, while momentum maintains an extra vector containing accumulated gradient information. Adam stores two values per parameter to track its first and second gradient moments. Adafactor reduces this requirement for large weight matrices by replacing the full second-moment array with smaller row and column statistics [5].

Methods that use curvature or matrix structure can have greater storage requirements. A full Hessian for a model with d parameters contains d^2 entries, making direct storage impractical for large networks. L-BFGS avoids this by retaining a limited history of parameter and gradient changes. K-FAC and Shampoo instead maintain smaller matrices associated with individual layers. Although these structured representations are much smaller than a full curvature matrix, they are not necessarily smaller than the diagonal statistics used by Adam [61], [6].

For example, a weight matrix of size $m \times n$ requires mn values to store one parameter-wise statistic. A structured approach that maintains separate square matrices for its rows and columns requires $m^2 + n^2$ values. This can be considerably smaller than the $(mn)^2$ entries of a full layer-wise curvature matrix, but it may still exceed the storage required by a coordinate-wise optimizer.

These comparisons must also distinguish optimizer-state memory from total training memory. Model parameters and gradients require storage regardless of the optimizer. Training also involves activations and temporary computational buffers, while mixed-precision and distributed implementations may maintain additional copies of parameters or optimizer states. Consequently, the memory requirements shown in Table II describe representative auxiliary states rather than the total memory consumed during training [24], [25].

Memory efficiency is therefore an important consideration, but it cannot determine an optimizer's overall effectiveness on its own. A method that stores less information may require more iterations, while another may use additional memory to construct more effective updates. The next subsection examines the computational cost of these updates and explains why memory savings do not always translate into faster training.

C. Arithmetic Cost, Communication, and Wall-Clock Time

Memory requirements represent only one aspect of an optimizer's computational cost. Even when two methods require similar amounts of storage, the work involved in computing their parameter updates can differ considerably. As a result, comparing optimization methods requires considering both the progress achieved in each iteration and the time needed to perform that iteration.

First-order methods such as SGD and Adam generally involve relatively inexpensive operations once the gradient has been computed. In contrast, curvature-aware methods may require additional calculations to estimate curvature or solve for an update direction. Structured methods also introduce overhead when maintaining and processing their matrix statistics. For example, Shampoo computes matrix inverse roots, while Muon performs repeated matrix multiplications to approximate orthogonalization [63], [24].

However, the mathematical complexity of an update does not directly determine its execution time. Modern GPUs can perform certain matrix operations efficiently through parallel processing. Other operations may be limited by the speed at which data can be transferred between memory and processing units. Consequently, the same optimizer may perform differently depending on the model architecture and hardware implementation.

These considerations become more important in distributed training, where the model and its computations are spread across multiple devices. Some optimizers require additional communication to synchronize gradient statistics or curvature estimates. Others can reduce this overhead by updating expensive information less frequently. Memory-management techniques such as ZeRO can also distribute optimizer states across devices, changing the practical memory requirements of training [25], [67], [68].

A meaningful comparison should therefore consider the total resources required to reach a specified training objective. An optimizer may require fewer iterations but spend considerably more time computing each update. Conversely, a method with simpler updates may achieve better overall efficiency despite slower progress per iteration. The distinction is illustrated by the SOAP results discussed in Section V-F, where reductions in iteration count and wall-clock time were not identical [13].

These factors show why optimizer efficiency cannot be judged from iteration count alone. The relevant question is how much computational effort is required to achieve a given level of optimization progress. However, even comparisons based on training time must be interpreted alongside theoretical convergence results, which may describe different objectives or rely on different assumptions. The next subsection examines how these guarantees should be compared.

D. Convergence Guarantees Must Match the Problem

The previous subsection showed that an optimizer's practical efficiency depends on more than the number of iterations required to reduce the training loss. A similar distinction is necessary when comparing theoretical convergence guarantees. Although these results describe how optimization methods progress toward a solution, they may measure different outcomes or rely on different assumptions.

For convex optimization problems, convergence is often measured by how quickly the objective value approaches

TABLE II: Representative optimizer auxiliary-state complexity. Here d is the total number of parameters, $W \in \mathbb{R}^{m\times n}$ is one matrix, and k is limited-memory history size. Counts exclude parameters, gradients, activations, sharding, master weights, and temporary buffers; they are not a universal peak-memory ranking.

Method	Typical persistent auxiliary state	Order	Main per-step overhead	Important qualification
GD / minibatch SGD	None (apart from iteration control)	$O(d^0)$	One gradient evaluation	Full and minibatch gradients have different data-access costs.
Heavy-ball momentum	Velocity vector	$O(d)$	Vector arithmetic	A momentum buffer can be stored at different precision.
Adam / AdamW	First- and second-moment arrays	$2d$	Elementwise operations	Optional master weights, sharding and implementation buffers are extra.
Lion	One momentum array	d	Elementwise sign and moments	Sign updates change tuning and effective step magnitude.
Adafactor (matrix, lean version)	Row and column second-moment sums	$m+n$	Factored reconstruction	Full configuration may maintain additional states.
Dense Newton	Full Hessian or factorization	$O(d^2)$	Second derivatives / linear solve	Hessian-free variants avoid dense storage but add inner products.
L-BFGS	k recent (s_i, y_i) pairs	$O(kd)$	Two-loop recurrence, possibly line search	Stochastic gradient noise can spoil curvature pairs.
K-FAC (one layer)	Two Fisher factors	$O(m^2+n^2)$	Factor updates, solves	Blocking, damping, inverse refresh, and communication vary.
Shampoo (one matrix)	Row/column accumulators	$O(m^2+n^2)$	Matrix inverse roots and multiplies	Practical implementations amortize and block factor operations.
Sophia	Momentum and diagonal-curvature estimates	$O(d)$	Occasional curvature estimation	Precise number of arrays depends on estimator and implementation.
Muon (eligible matrices)	Momentum, plus temporary workspaces	$O(d)$	Newton–Schulz matrix products	Auxiliary AdamW state for ineligible parameters may remain.

its minimum. Under suitable conditions, accelerated methods such as Nesterov's can achieve better theoretical convergence rates than ordinary gradient descent [28], [18]. However, deep neural networks generally involve non-convex objectives, where finding the global minimum cannot be guaranteed by standard gradient-based methods. In this setting, convergence analysis often focuses on reaching a stationary point where the gradient is sufficiently small.

Stochastic optimization introduces another difficulty because minibatch gradients contain sampling noise. As discussed in Section II-F, this uncertainty affects the progress expected from each update. Convergence guarantees for SGD therefore depend on assumptions about gradient variance and learning-rate selection [3], [26]. These results cannot be compared directly with deterministic guarantees without accounting for the additional uncertainty.

The same caution applies to curvature-aware methods. Newton's local quadratic convergence describes its behavior near a suitable minimum, while other approaches provide guarantees under different smoothness or curvature assumptions. Similarly, theoretical results for adaptive and structured optimizers are often established for particular classes of problems. A guarantee derived for convex optimization does not automatically apply to non-convex neural-network training [1], [33], [6].

Consequently, convergence rates should only be compared when they describe the same measure of progress under comparable assumptions. Even then, a stronger theoretical rate does not necessarily imply shorter training time, since the cost of computing each update may differ substantially. A meaningful evaluation must therefore consider both the mathematical guarantees and the practical conditions under which an optimizer is used.

Beyond convergence, optimization methods may also behave differently in terms of stability and their sensitivity to gradient noise. These properties can influence the training process even when two methods have similar convergence guarantees. The next subsection examines these differences and distinguishes optimization performance from generalization to unseen data.

E. Stability, Noise, and Generalization Are Different Questions

Convergence guarantees describe whether an optimization method approaches a particular solution or stationary point under certain conditions. However, they do not fully explain how the optimizer behaves throughout training. An optimization method may make overall progress while experiencing substantial fluctuations in its parameter updates. Understanding this behavior requires considering stability and the effects of gradient noise.

As discussed in Section II, the stability of gradient descent depends on the relationship between the learning rate and the local curvature of the loss function. A large learning rate can cause updates to oscillate or become unstable, particularly in directions with high curvature. However, the loss landscape of a neural network changes as its parameters are updated. Recent studies of *edge-of-stability* behavior show that neural networks can sometimes continue making progress even when their local

curvature exceeds classical stability thresholds [69]. This does not mean that large learning rates are always safe, but rather that local stability analysis does not fully describe the behavior of nonlinear training.

Gradient noise introduces another consideration. Since SGD estimates gradients from minibatches, its updates fluctuate according to the sampled data. This randomness may help the optimizer move away from certain saddle points, where the gradient is small but the parameters have not reached a minimum [10], [19]. However, the same noise can interfere with methods that rely on accurate curvature estimates. Its effects therefore depend on both the optimization method and the region of the loss landscape being explored.

Stability and convergence should also be distinguished from *generalization*. An optimizer may reduce the training loss efficiently without producing a model that performs better on unseen data. Research has examined whether properties of the loss landscape, such as the sharpness of a minimum, are related to generalization. However, this relationship is not straightforward because measures of sharpness can depend on how the network is parameterized [17]. Studies comparing batch sizes and optimization methods have also reported different generalization behaviors under different training conditions [15], [70], [16], [71].

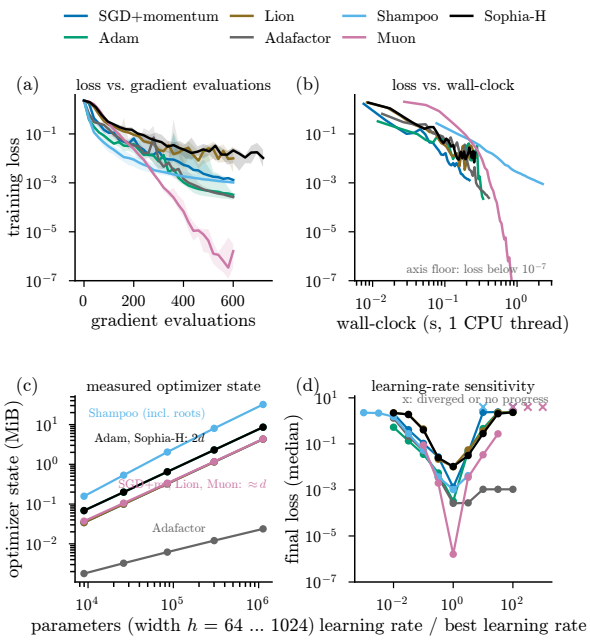
These distinctions show why optimizer performance cannot be evaluated through training loss alone. A reliable comparison should also examine the stability of the training process and the model's performance on held-out data. The next subsection translates these considerations into a reproducible evaluation protocol for comparing optimization methods under consistent experimental conditions.

F. A Reproducible Comparison Protocol

The previous discussion highlighted several factors that influence optimizer performance. To understand whether one method offers a meaningful advantage over another, these factors must be considered within a consistent experimental framework. Otherwise, differences in training conditions may be mistaken for improvements caused by the optimizer itself.

A useful evaluation can begin with a simple quadratic loss function whose curvature is known. As discussed in Section II-D, comparing an axis-aligned valley with a rotated valley helps reveal how different methods respond to relationships between parameter directions. This provides a controlled setting for examining the behavior of gradient descent and curvature-aware methods. It also makes the limitations of diagonal scaling easier to observe.

The comparison should then extend to a neural-network training task. To ensure fairness, the methods should use the same model architecture and dataset split. They should also be evaluated under comparable training budgets and random initializations. Hyperparameters must be tuned with similar effort, since poor tuning can make an otherwise effective optimizer appear unsuitable. Repeating experiments with multiple random seeds helps determine whether observed improvements are consistent.



Measured runs: MLP 64-128-128-10 (ReLU), sklearn digits (1,437 train), batch 64, 600 updates, 5 seeds, fp32, CPU (Intel Core Ultra 9 285H), PyTorch 2.11; 9-point LR grid per optimizer; (a), (b) at each optimizer's best LR.

Fig. 9: Recommended evaluation dashboard. Matching compute, data, initialization, and tuning effort is necessary before interpreting empirical optimizer comparisons.

Performance should be evaluated using more than one measure. Training loss indicates optimization progress, while wall-clock time captures the computational cost required to achieve it. Memory consumption provides another important measure, particularly for large models. Learning-rate sensitivity should also be examined because some optimizers may perform well only within a narrow range of settings. Figure 9 brings these comparisons together through a common evaluation dashboard.

For methods that exploit matrix structure, controlled experiments can provide additional insight. Comparing how different optimizers transform the same matrix-shaped gradient helps isolate the effect of their update rules. Such experiments can illustrate differences between matrix preconditioning and orthogonalization, although they cannot establish which method will perform best across neural-network training tasks.

A reproducible comparison should therefore report both the observed training results and the conditions under which they were obtained. This allows performance differences to be interpreted in relation to computational cost and optimizer design. It also provides a more reliable basis for understanding when the additional complexity of an optimization method is justified.

G. A Compact Cross-Family Summary

The comparison of optimization methods reveals that no single approach is consistently preferable across all training problems. Each method is designed to address particular limitations, and its effectiveness depends on

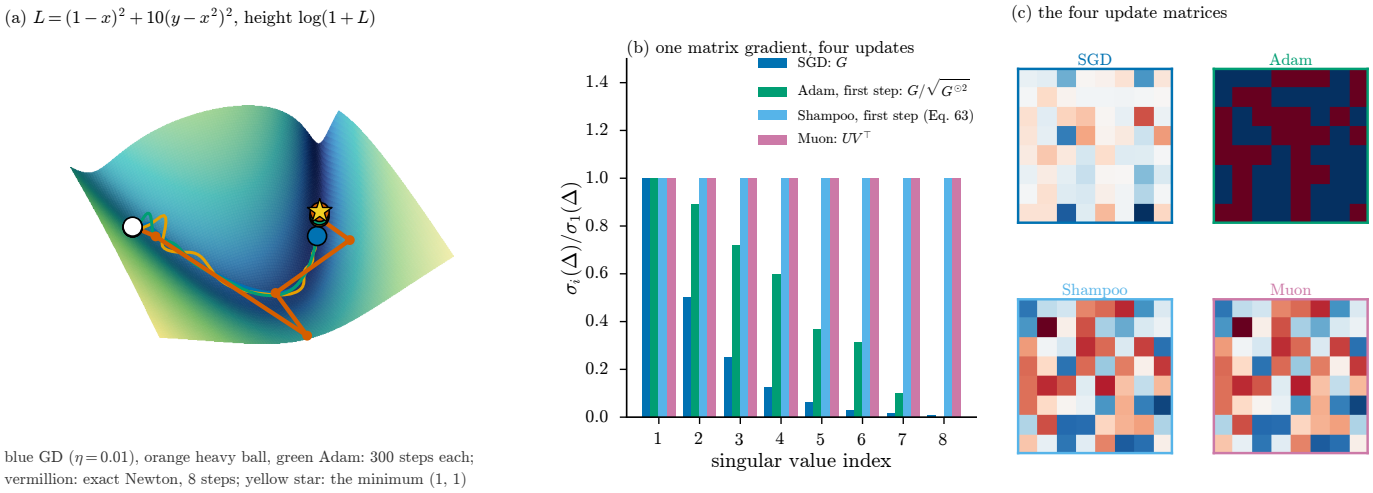


Fig. 10: The analytic comparisons recommended in Section VI-F. (a) GD, heavy-ball momentum, Adam, and exact Newton on one analytic objective, $L = (1-x)^2 + 10(y-x^2)^2$, drawn as balls rolling over the 3D landscape (height $\log(1+L)$) from the same start $(-1.2, 1)$ (white ball) toward the minimizer $(1, 1)$ (yellow star). (b) Singular values of the update that each method builds from one controlled 8×8 gradient G with singular values $4, 2, 1, \ldots, 0.03$. SGD keeps the spectrum of G; Adam's first step $G/\sqrt{G^{\odot 2}}$ is elementwise and does not flatten it; one Shampoo step from Eq. (37) with no accumulated history equals $UV^\top$, the Muon direction of Eq. (40). (c) The four update matrices (colour scale normalized per matrix). With statistics accumulated over many steps, Shampoo and Muon differ. A toy example; it does not establish superiority in deep learning.

TABLE III: What each family answers and what it does *not* establish by itself.

Family	Main strength	Not automatically implied
SGD/momentum	Simple, low auxiliary overhead	Fastest wall-clock convergence
Adam/AdamW	Coordinate adaptation	Hessian approximation or global optimum
Adafactor/Lion	Lower auxiliary state in selected regimes	Better loss on every task
Newton/L-BFGS	Local or secant curvature	Stable global non-convex progress
K-FAC	Structured Fisher approximation	Exact Hessian or cheap communication
Shampoo/SOAP	Tensor geometry and rotated adaptation	Lower total memory than all diagonal methods
Sophia	Estimated diagonal Hessian and clipping	Universal reported speedup
Muon	Orthogonalized hidden-weight updates	A second-order Hessian guarantee

whether those limitations are significant in the task being considered.

First-order methods remain attractive because their updates are relatively simple and computationally efficient. Adaptive methods improve how gradients are used, but their additional statistics introduce memory costs. Curvature-aware methods can construct more informed updates by accounting for the local behavior of the loss. However, their benefits must justify the expense of obtaining and processing curvature information. Structured and matrix-aware methods offer further alternatives by exploiting the organization of neural-network parameters, although they introduce their own computational requirements.

As summarized in Table III, these differences make it difficult to establish a universal ranking of optimizers. A method that performs well on a small model may become expensive at a larger scale. Similarly, an approach that provides useful updates for one parameter structure may offer little advantage for another. Performance must therefore be interpreted in relation to the training problem and the available computational resources [3], [7], [12], [13].

The central conclusion is that optimizer selection involves balancing the quality of parameter updates against their practical cost. This also highlights several questions that remain unresolved. In particular, it is not always clear when more sophisticated geometric information produces enough improvement to justify its overhead. These challenges motivate the open research directions discussed in Section VII.

VII. Open Problems and Future Directions

A. When Does Better Geometry Repay Its Cost?

The previous section highlighted a central trade-off in neural-network optimization. Methods that use curvature or matrix structure can produce more informed parameter updates, but obtaining this information requires additional resources. An important research question is whether these improvements are large enough to justify their computational cost.

TABLE IV: Every optimizer in this survey, one row each: the update it takes, what it remembers between steps, the knobs that must be tuned, and what its stored quantity really is. Equation numbers refer to this survey; Table II gives the memory cost of each state.

Method	Update (one step)	Remembers	Knobs to tune	What the remembered quantity is
First-order methods: only gradients (Section III)				
GD / SGD	$\theta \leftarrow \theta - \eta\, g$ (full or minibatch g), Eqs. (13), (14)	nothing	η, batch size	—
Heavy ball	$v \leftarrow \mu v + g,\ \theta \leftarrow \theta - \eta v$, Eqs. (16), (17)	velocity v	η, μ	running sum of past gradients
Nesterov	gradient taken at the look-ahead point $\theta_t + \mu(\theta_t - \theta_{t-1})$, Eqs. (18), (19)	last iterate	η, μ	where the velocity is about to go
AdaGrad	$\theta_j \leftarrow \theta_j - \eta\, g_j/(\sqrt{s_j} + \varepsilon),\ s_j \mathrel{+}= g_j^2$, Eqs. (20), (21)	s	η	total squared gradient per coordinate
RMSProp	same, but s is an exponential average of g^2, Eq. (22)	s	η, ρ	recent squared gradient per coordinate
Adam	$\theta \leftarrow \theta - \eta\, \hat{m}/(\sqrt{\hat{v}} + \varepsilon)$, Eqs. (23)–(25)	m, v	η, β_1, β_2	m: smoothed direction; v: squared-gradient scale (not a Hessian)
AMSGrad	Adam with $\bar{v} = \max(\bar{v}, v)$ in the denominator, Eq. (26)	m, v, $\bar{v}$	as Adam	largest scale seen so far
AdamW	Adam step, plus every weight shrunk by $1 - \eta\lambda$, Eq. (28)	m, v	as Adam, $+\lambda$	weight decay kept outside the adaptive scale
Adafactor	Adam-like step with v rebuilt from row and column sums, [5]	row sums r, column sums c	η, decay	a rank-one picture of the squared-gradient scale
Lion	$\theta \leftarrow \theta - \eta\, \mathrm{sign}$(momentum blend), Eqs. (29)–(31)	m	η, β_1, β_2, λ	momentum; only its sign is used
Second-order methods: curvature of the loss (Section IV)				
Newton	solve $H\Delta = -g,\ \theta \leftarrow \theta + \Delta$, Eq. (32)	(the Hessian)	damping	the exact local curvature
Damped / trust region / cubic	Newton with $H + \lambda I$, a radius $\|\Delta\| \le r$, or a $\|\Delta\|^3$ penalty, Eq. (33), [43], [44]	—	λ, r, ρ	curvature plus a safety limit on the step
BFGS / L-BFGS	$\Delta = -Cg$ with C updated from (s, y) pairs, Eq. (34), [51]	C, or the last k pairs	k, line search	curvature learned from gradient changes
Hessian-free	solve $(G + \lambda I)\Delta = -g$ by CG, using only products Gv, Alg. 1	—	CG iterations, λ	curvature used through products, never stored
Natural gradient	$\Delta = -\eta F^{-1} g$, Eq. (35)	(the Fisher)	η, damping	how much a step changes the predictions
Structured and matrix-aware methods (Section V)				
K-FAC	step $\propto S^{-1}\,(\nabla_W L)\,A^{-1}$, Eq. (36)	A, S per layer	η, damping, refresh	two small factors of the Fisher block
Shampoo	$\Delta W = -\eta\, L^{-1/4}\, G\, R^{-1/4}$, Eq. (37)	L, R per matrix	η, ϵ, refresh	row and column gradient statistics
SOAP	Adam in the rotated basis $U^\top G V$ of Shampoo, Eq. (38)	Adam moments, U, V	as Adam, refresh	per-coordinate scales in a better basis
Sophia	$\Delta\theta = -\eta\, \mathrm{clip}(m/\max\{h, \epsilon\}, \pm\rho)$, [12]	m, h	η, ρ, how often h is estimated	a diagonal Hessian estimate [12]
Muon	$\Delta W = -\eta UV^\top$ from the momentum $M = U\Sigma V^\top$, Eqs. (39), (40)	momentum M	η, β, iterations	no curvature: singular values set to one

This question is particularly relevant to structured methods such as K-FAC and Shampoo. Both use matrix information to improve parameter updates, but their efficiency depends on how accurately that information is approximated and how frequently it must be updated. Recomputing matrix statistics more often may improve their usefulness, but it also increases training overhead. Updating them less frequently reduces the cost, although the information may become less representative of the current loss landscape [61], [63].

A similar trade-off appears in Hessian-free optimization, where each parameter update may require several internal calculations. Muon presents a different challenge because its matrix orthogonalization can be efficient on modern accelerators, yet its cost depends on the shapes of the weight matrices being updated. Since Muon may also require another optimizer for certain parameters, evaluating its overall efficiency requires considering the complete training process [7].

These examples suggest that the value of additional geometric information cannot be determined from the update rule alone. It depends on how effectively that information improves optimization relative to the time and resources needed to obtain it. Future research should therefore investigate when more informative updates provide meaningful reductions in total training cost, particularly as model

TABLE V: Every convergence or stability result quoted in this survey, grouped by kind, with the setting it needs and the quantity it measures. Results in different groups measure different things and cannot be ranked by their power of T (Section VI-D).

Result	Holds when	What it measures	Each step uses	Where
Convex problems: rates for the loss value				
GD: error $O(1/T)$	smooth convex loss	$L(\theta_T) - L_*$	full gradient	[18]
Nesterov: error $O(1/T^2)$	smooth convex loss, the right coefficient schedule	$L(\theta_T) - L_*$	full gradient	[28]
GD rate set by κ	strongly convex quadratic	error per step	full gradient	Sec. II-D
Non-convex problems: rates for the gradient size				
GD: $O(\epsilon^{-2})$ steps to $\|g\| \le \epsilon$	β-smooth, bounded below, $\eta \le 1/\beta$	smallest $\|g_t\|^2$ seen	full gradient	Eq. (15)
SGD: $O(\epsilon^{-4})$ steps	as above, plus unbiased gradients with variance $\le \sigma^2/B$	average $\mathbb{E}\|g_t\|^2$	minibatch gradient	Eqs. (46)–(49)
Adam-type methods	extra conditions (bounded gradients, controlled step sizes)	stationarity	minibatch gradient	[33], [34]
Local results: near one point				
Newton: quadratic convergence	close to a minimum with $H \succ 0$, Lipschitz Hessian	$\|\theta_t - \theta^*\|$	Hessian and a solve	Sec. IV-B
BFGS: superlinear convergence	close to a minimum, good line search	$\|\theta_t - \theta^*\|$	gradients only	[51]
Stability: $\eta < 2/\lambda_{\max}$	fixed quadratic (local model)	does the error shrink?	full gradient	Eq. (9)
Heavy ball: $0 < \eta\lambda < 2(1+\mu)$	1-D quadratic, $0 \le \mu < 1$	does the error shrink?	full gradient	Fig. 4(c)
Preconditioned GD: $\eta < 2/\lambda_{\max}(K)$	fixed quadratic, fixed positive-definite M	does the error shrink?	gradient and M	Eq. (58)
Online and stochastic convex results: regret				
AdaGrad; Adam's failure and AMSGrad	online convex problems	regret, convergence	minibatch gradient	[29], [32]
Shampoo	stochastic convex problems	regret	gradient and matrix roots	[6]
Global safeguards, saddles and geometry				
Trust region, cubic regularization	stated smoothness, access to the Hessian	certain stationarity criteria	Hessian and a subproblem	[44], [45]
Random perturbations escape strict saddles	the cited papers' assumptions	approximate second-order stationarity	gradient and noise	[10], [19]
Muon's step is steepest under $\|\cdot\|_2$	one linearized step, budget $\|\Delta\|_2 \le \eta$	decrease of the linear model (not convergence)	gradient matrix	[66]

sizes and hardware configurations change [25].

Understanding this trade-off is essential for developing optimization methods that are both mathematically effective and practical at scale. However, computational cost is not the only challenge. The quality of the geometric information itself can also be affected by stochastic gradients and non-convex loss landscapes, as discussed in the next subsection.

B. Curvature Under Noise and Non-Convexity

The previous subsection examined the computational cost of using geometric information during optimization. However, the usefulness of this information also depends on its accuracy. In deep neural networks, curvature changes as the parameters are updated, making it difficult to maintain an accurate estimate throughout training. This challenge becomes greater when curvature is estimated from minibatches rather than the full dataset.

As discussed in Section II, minibatch sampling introduces noise into gradient estimates. Similar uncertainty can affect curvature estimates, particularly when only a small subset of the training data is used. Consequently, an optimizer may construct an update based on curvature information that does not accurately represent the full training objective. This can reduce the benefit of curvature-aware methods or make their updates unstable.

The non-convex nature of neural-network loss functions introduces further difficulties. The Hessian may contain negative eigenvalues, while simpler approximations may fail to capture important relationships between parameters. Moreover, the Fisher information matrix and gradient-based statistics do not generally represent the same information as the Hessian [58]. Their usefulness therefore depends on how well they capture the properties needed to guide optimization.

Methods such as damping and clipping can help control the effects of inaccurate curvature estimates. However, these safeguards introduce additional choices that influence optimizer behavior. Stronger regularization may improve stability but can also reduce the benefit of using curvature information.

An important direction for future research is to de-

TABLE VI: The matrices that are all called "curvature," answered one question at a time. Sizes are for one $m \times n$ weight matrix with mn parameters (Sections IV-H and V-H).

Matrix	Built from	Never negative?	Is it the Hessian?	Numbers stored	Used by
Hessian H	second derivatives of the loss, Sec. IV-H	×	yes	$(mn)^2$	Newton, trust region, cubic regularization
Gauss–Newton G	$J^\top H_\ell J$, Sec. IV-H	✓ (when $H_\ell \succeq 0$)	no: H with its second term dropped	$(mn)^2$	Hessian-free (as products Gv)
Fisher F	outer products of the score, labels drawn from the model, Sec. IV-H	✓	no; equal only in special likelihood cases	$(mn)^2$	natural gradient, K-FAC
Empirical Fisher	outer products of the gradient with the true labels	✓	no; not F, G or H in general	$(mn)^2$	gradient outer-product estimates
K-FAC factors	$A = \mathbb{E}[aa^\top]$, $S = \mathbb{E}[\delta\delta^\top]$, Eq. (36)	✓	no: $A \otimes S$ approximates the Fisher block	$n^2 + m^2$	K-FAC
Shampoo factors	accumulated $GG^\top$ and $G^\top G$, Eq. (37)	✓	no: gradient statistics	$m^2 + n^2$	Shampoo, SOAP
Adam's v	average of squared gradients, Eq. (24)	✓	no: $\mathbb{E}[g_j^2]$, not $\partial^2 L/\partial\theta_j^2$	mn	Adam, AdamW
Sophia's h	averaged $z \odot (Hz)$ [12]	×	its diagonal, estimated without bias	mn	Sophia (with clipping)
(Muon)	nothing stored; $UV^\top$ of the momentum, Eq. (40)	—	no: a norm geometry, not a metric matrix	mn	Muon

termine how accurate curvature estimates need to be to provide meaningful improvements during training. This requires studying their behavior under different levels of gradient noise and across loss landscapes with varying curvature. It is also important to understand how the quality of these estimates changes as optimization progresses.

Addressing these questions could help establish when approximate curvature information is reliable enough to justify its use. However, not all modern optimization methods depend on curvature estimates. Matrix-aware approaches such as Muon introduce different geometric transformations, raising theoretical questions that cannot be answered through conventional curvature analysis alone.

C. Matrix-Aware Methods Need Their Own Theory

The previous subsection examined the challenges of using curvature information in non-convex optimization. However, some modern methods improve parameter updates without explicitly estimating curvature. Muon is an example of this approach, as it uses matrix orthogonalization to modify the direction of momentum-based updates [7].

As discussed in Section V-G, Muon changes the singular values of an update matrix while preserving its singular directions. This operation can be understood through spectral-norm geometry, which provides a mathematical interpretation of why the transformed update may be useful [66]. However, this interpretation alone does not fully explain Muon's behavior during neural-network training.

In practice, Muon's updates are influenced by momentum and stochastic gradients. Its orthogonalization is also computed approximately rather than through an exact singular value decomposition. These factors can affect the optimization trajectory, making it difficult to draw general conclusions from the idealized mathematical model. Further theoretical work is needed to understand how these mechanisms interact and under what conditions they lead to stable convergence.

Another open question concerns which neural-network parameters benefit from matrix-aware updates. Methods such as Muon are primarily applied to suitable weight matrices, while other parameters are often optimized using AdamW [7]. Understanding why certain parameter structures benefit more than others could help guide the development of more effective hybrid optimization methods.

These questions also extend to methods such as Shampoo and SOAP. Although they use matrix transformations, their underlying mechanisms differ from Muon's orthogonalization. A meaningful theoretical comparison must therefore account for the information each method uses rather than treating all matrix-aware algorithms as equivalent second-order methods [7], [13], [66].

Developing such theory could clarify when matrix geometry offers advantages over conventional adaptive scaling. It may also help explain how optimizer behavior depends on the representation of model parameters. This leads to the next research direction, which examines how changes in parameterization affect optimization.

D. Invariance and Parameterization

The previous subsection highlighted the importance of understanding how optimizer behavior depends on the structure of neural-network parameters. A related challenge arises from the way these parameters are represented. In some neural networks, different parameter

values can produce the same predictions. However, an optimization method may behave differently under these equivalent representations.

This occurs because many optimizers determine their updates using the numerical values of parameters and gradients. For example, rescaling the weights of certain layers can leave the network's output unchanged while altering the gradients used during training. As a result, gradient descent and coordinate-wise adaptive methods may follow different optimization trajectories even though the underlying predictive model is equivalent.

This motivates the study of *invariance*, which describes whether an optimization method behaves consistently under changes in parameter representation. Natural-gradient methods address this issue by measuring updates according to their effect on the model's predicted distributions rather than relying only on distances between parameter values [59], [57]. Structured and matrix-aware methods introduce other geometric properties that may affect their sensitivity to parameterization.

Understanding these differences is particularly important for modern neural networks, where normalization and parameter scaling can influence the optimization process [72], [17]. However, greater invariance does not necessarily guarantee faster training or better generalization. Its practical value depends on how the resulting update behavior interacts with the model and training conditions.

Future research should therefore examine how optimizer performance changes under equivalent parameterizations of the same network. Such comparisons could help distinguish improvements arising from an optimizer's underlying mathematical principles from those that depend on a particular model representation. This would also contribute to more reliable evaluations of optimization methods, motivating the benchmarking considerations discussed next.

E. Toward Fair, System-Aware Optimizer Benchmarks

The challenges discussed in this section highlight the difficulty of evaluating optimization methods under different training conditions. An optimizer may perform well on one model but offer fewer advantages when the architecture or computational resources change. This makes it difficult to determine whether reported improvements reflect the optimizer's design or the specific conditions of the experiment.

A central problem is that optimizer studies do not always measure efficiency in the same way. Some emphasize the number of iterations required to reach a target loss, while others report training time or computational cost. These measures provide different perspectives on performance. As discussed in Section VI, a method that requires fewer iterations may still be slower if each update involves expensive calculations.

Future benchmarks should therefore evaluate optimization progress alongside the resources required to achieve it. Training time and peak memory consumption are particularly important for large neural networks. The number of processed examples or tokens can also help clarify differences in training efficiency. Reporting these measures together would provide a more complete picture of an optimizer's practical value [67], [68].

Fair comparisons also require consistent experimental conditions. Methods should be evaluated using comparable training budgets and hyperparameter-tuning effort. Their implementations should be documented clearly, since choices involving numerical precision and hardware can substantially influence performance. Making these details available would improve reproducibility and help researchers understand why results differ across studies.

More consistent benchmarking would make it easier to identify the conditions under which particular optimization methods offer meaningful advantages. It would also support research on the trade-offs between update quality and computational cost. Ultimately, progress in neural-network optimization should be assessed not only by improvements in mathematical update rules, but also by their demonstrated benefits in practical training.

VIII. Conclusion

Training deep neural networks requires optimization methods that can efficiently reduce the loss while accounting for the challenges of non-convex objectives. This survey examined how different optimizers address these challenges by changing the way gradients are used to construct parameter updates. Although the methods differ in their mathematical foundations, they share the objective of improving the optimization process.

In addressing RQ1, the survey showed how first-order methods extend standard gradient descent. Momentum improves the use of previous gradient directions, while adaptive methods adjust parameter updates based on historical gradient statistics. More recent approaches also seek to reduce memory requirements and simplify update calculations. These developments demonstrate how gradient-based optimization can be improved without explicitly estimating the curvature of the loss function.

In addressing RQ2, the survey examined methods that incorporate additional information into the update process. Second-order methods use curvature to account for how the gradient changes, while natural-gradient methods consider the sensitivity of the model's predicted distributions. Structured and matrix-aware approaches make use of neural-network parameter organization to construct updates that would otherwise be computationally expensive. Importantly, these methods do not all estimate the same mathematical quantities, even when their update rules involve similar matrix operations.

The comparison also highlighted that more informative updates do not necessarily lead to more efficient training. Their advantages must be considered alongside computational cost and memory requirements. Theoretical convergence guarantees provide useful insights, but their practical relevance depends on the assumptions under which they hold.

Overall, the survey shows that understanding an optimizer requires examining both the information it uses

and how that information influences parameter updates. Rather than identifying a universally superior method, this perspective provides a basis for evaluating optimization approaches according to the requirements of the training problem and the resources available.

Appendix A
Worked Mathematical Connections

A. Diagonal Scaling and a Rotated Quadratic

For $L(\theta) = \frac{1}{2}\theta^\top H\theta$ and diagonal positive definite D, the preconditioned step $\theta^+ = \theta - \eta D^{-1}H\theta$ corresponds, after the change of variables $z = D^{1/2}\theta$, to $z^+ = (I - \eta D^{-1/2}HD^{-1/2})z$. Thus convergence is governed by the *symmetrically preconditioned* matrix $D^{-1/2}HD^{-1/2}$. If H is diagonal and $D = H$, a single unit step solves the quadratic. If H has off-diagonal couplings, merely choosing $D = \mathrm{diag}(H)$ need not remove them. This formalizes the geometric diagrams in Fig. 3 [1], [29].

For a concrete 45° rotation, let

$$Q = \frac{1}{\sqrt{2}}\begin{bmatrix}1 & -1\\ 1 & 1\end{bmatrix}, \qquad H = Q^\top \begin{bmatrix}100 & 0\\ 0 & 1\end{bmatrix} Q. \tag{41}$$

Direct multiplication gives $H_{11} = H_{22} = 50.5$ and off-diagonal entries with absolute value 49.5. The diagonal contains no information about the large coupling. This example shows why full or structured transformations can offer information unavailable to purely diagonal methods, even though they need more computation.

B. Bias Correction as a Finite Geometric Series

With constant gradient g and $m_0 = 0$, the first-moment recursion $m_t = \beta m_{t-1} + (1-\beta)g$ gives

$$m_t = (1-\beta)\sum_{k=0}^{t-1}\beta^k g = (1-\beta^t)g. \tag{42}$$

The correction $\hat{m}_t = m_t/(1-\beta^t)$ therefore returns g in this idealized stationary case. For changing stochastic gradients, the interpretation is more subtle, but the geometric-series calculation explains why Adam needs separate corrections for its two exponential averages [4].

C. The Polar Factor and the Nuclear Norm

Write $G = U\Sigma V^\top$. For any update Δ with $\|\Delta\|_2 \le \eta$, trace duality gives $|\langle G, \Delta\rangle_F| \le \|G\|_*\|\Delta\|_2$, where $\|G\|_* = \sum_i \sigma_i(G)$ is the nuclear norm. Choosing $\Delta = -\eta UV^\top$ attains the lower bound along the singular directions, showing why the polar factor is the steepest *linearized* direction for a spectral-norm budget. This does not establish a global neural-network convergence theorem for approximate Newton–Schulz-orthogonalized momentum [7], [66].

Appendix B
Proposed Figure and Reproducibility Checklist

All boxed figure slots in this document are intentional blanks. Figures 1–9 specify the concepts to illustrate; they contain no claimed experimental output. Before submission with finalized figures, identify whether each panel is (i) an analytic mathematical plot, (ii) a replication of a published figure with proper attribution, or (iii) a new experiment. Analytic bowls, rotated quadratics, momentum trajectories, SVD singular-value plots, and theoretical memory curves can be generated from their explicitly stated equations. Empirical neural-network training curves must come from actual code runs and should report architecture, dataset, seeds, learning rates, precision, hardware, stopping criteria, and optimizer implementation.

For plots of optimization loss, also specify whether the horizontal axis represents parameter updates, complete data passes, examples/tokens processed, gradient evaluations, or wall-clock time. For memory figures, indicate whether the values include weights, gradients, master weights, optimizer state, factor buffers, activation checkpoints, and distributed sharding. Without these labels, a visual comparison can be technically correct in isolation but misleading to the reader.

Appendix C
Additional Worked Derivations and Practical Diagnostics

A. Step-by-Step Comparison on a Rotated Valley

Consider again the rotated quadratic $L(\theta) = \theta^\top H\theta/2$ with

$$H = \begin{bmatrix}50.5 & -49.5\\ -49.5 & 50.5\end{bmatrix}, \qquad \theta_0 = \begin{bmatrix}1\\ 0\end{bmatrix}. \tag{43}$$

Its gradient at the starting point is $g_0 = H\theta_0 = (50.5, -49.5)^\top$ and the loss is $L(\theta_0) = 25.25$. All methods see the *same gradient*. Their different steps reveal what is added by the optimizer, not what is added by the objective.

First, fixed-step GD with $\eta = 0.01$ produces

$$\theta_1^{\mathrm{GD}} = \theta_0 - 0.01 g_0 = \begin{bmatrix}0.495\\ 0.495\end{bmatrix}. \tag{44}$$

Direct substitution gives $L(\theta_1^{\mathrm{GD}}) = 0.5(0.495, 0.495)H(0.495, 0.495)^\top = 0.245025$. The GD step happens to move close to the low-curvature eigenvector because the chosen learning rate cancels the large-curvature eigencomponent. This is a specific, calculated example, not a claim that GD always makes such progress in one step.

Next, use the *exact diagonal of the Hessian* as a preconditioner: $D = \mathrm{diag}(H) = 50.5I$. At learning rate $\eta = 1$, the update becomes

$$\theta_1^{\mathrm{diag}} = \theta_0 - D^{-1}g_0 = \begin{bmatrix}0\\ 49.5/50.5\end{bmatrix} \approx \begin{bmatrix}0\\ 0.9802\end{bmatrix}. \tag{45}$$

Even though D contains the *exact* diagonal curvature, it discards the off-diagonal interaction -49.5 and therefore

TABLE VII: Where every figure comes from, in the terms of Appendix B: (i) analytic mathematical plot, (ii) replication of a published figure, or (iii) new experiment. No figure is a replication. In the 3D figures the surface is the exact loss and every ball path is the exact sequence of iterates; only the drawn height is compressed where the figure says so. Scripts are in the folder `survey-v3/`.

Fig.	Type	What is drawn	Computed from	Script
1	diagram	map of Sections III–V	the survey's structure; no data	`tex/fig1_map.tex`
2	(i) analytic	(a)–(d) bowl, valleys and saddle in 3D, with GD and SGD balls; (e)–(i) what each kind of stationary point looks like	$\frac{1}{2}\theta^\top H\theta$ for the four H shown; Sec. II-E definitions; seeded gradient noise for SGD	`figs/make_figs3d.py`
3	(i) analytic	the 100:1 valley, aligned and rotated, with GD, diagonal and Newton balls	Eqs. (8), (41), (56)	`figs/make_figs3d.py`
4	(i) analytic	(a) SGD and heavy-ball balls in a narrow valley; (b) a 1-D ball over time; (c) where it settles	Eqs. (16)–(17); seeded gradient noise in (a)	`figs/make_figs3d.py`
5	diagram	Adam's two streams and its memory	Eqs. (23)–(25); Table II	`tex/fig5_adam.tex`
6	(i) analytic	Newton's quadratic model in 1-D and one GD vs Newton step on a 3D bowl	Sections IV-A, IV-C; Eq. (32)	`figs/make_figs3d.py`
7	diagram	one layer, its gradient and the Kronecker factors, to scale for $m = 4$, $n = 3$	Eq. (36)	`tex/fig7_kfac.tex`
8	(i) analytic	singular values before and after Newton–Schulz	SVD of a seeded 16×12 matrix; Eq. (50); Muon's quintic coefficients [7]	`figs/make_figs.py`
9	(iii) experiment	loss, time, memory and learning-rate sensitivity of seven optimizers	measured runs, Table VIII	`experiments/`
10	(i) analytic	four optimizers on one 3D landscape; four updates from one matrix gradient	Rosenbrock-type loss; Eqs. (25), (37), (40)	`figs/make_figs3d.py`
11, 14	(i) analytic	first steps on the rotated valley; GD, Newton and damped Newton at a saddle, in 3D	Appendices C-A and D-A	`figs/make_figs3d.py`
12, 13	(i) analytic	the SGD bound against η and T; the scalar Newton–Schulz map	Eqs. (47)–(49) (illustrative constants); Eq. (51)	`figs/make_figs.py`

does not solve the problem. The loss is about 24.26, only modestly below the starting 25.25. In contrast, full Newton solves $H\Delta = -g_0$, so $\Delta = -\theta_0$ and $\theta_1^{\text{Newton}} = 0$. This numerical contrast explains precisely why a diagonal Hessian or adaptive coordinate scale is not equivalent to full inverse-Hessian preconditioning. It also shows that no one-step ranking is meaningful without a fair selection of learning rates for each method [1], [29].

The same construction can be used to generate a useful figure: draw the ellipses of L, mark the common start, and show one step under GD, diagonal scaling, and Newton. Adding arrows for their subsequent steps makes the effect of conditioning visible. Every plotted quantity is determined by the explicit 2×2 matrix above and requires no empirical neural-network training.

B. An SGD Bound With an Explicit Step-Size Choice

Let $\Delta_0 = L(\theta_0) - L_*$, and assume L is β-smooth, bounded below, and minibatch gradients are conditionally unbiased with conditional variance at most σ^2/B. For a constant step $\eta \le 1/\beta$, Eq. (11) simplifies to

$$\mathbb{E}_t[L_{t+1}] \le L_t - \frac{\eta}{2}\|g_t\|^2 + \frac{\beta\eta^2\sigma^2}{2B}. \tag{46}$$

Take full expectations, sum for T iterations, use the lower bound L_*, and divide by $\eta T/2$. The result is

$$\frac{1}{T}\sum_{t=0}^{T-1}\mathbb{E}\|g_t\|^2 \le \frac{2\Delta_0}{\eta T} + \frac{\beta\eta\sigma^2}{B}. \tag{47}$$

The first term decreases when the step size grows; the second increases. If we temporarily ignore the constraint $\eta \le 1/\beta$, differentiate the right-hand side with respect to η:

$$-\frac{2\Delta_0}{\eta^2 T} + \frac{\beta\sigma^2}{B} = 0. \tag{48}$$

Thus a balanced choice is

$$\eta_{\text{bal}} = \sqrt{\frac{2\Delta_0 B}{\beta\sigma^2 T}}, \qquad \eta = \min\left\{\frac{1}{\beta}, \eta_{\text{bal}}\right\}. \tag{49}$$

Substitution into Eq. (47) gives a bound of order $T^{-1/2}$ in the stochastic regime for the *average expected squared gradient norm*, provided the bounded-variance and independence assumptions are valid. This is why an *ϵ-gradient-norm* objective leads, through this particular worst-case bound, to an $O(\epsilon^{-4})$ iteration dependence. It is not an empirical speed prediction and should not be confused with an $O(1/T)$ convex function-value statement [3], [26].

C. How a Matrix Iteration Can Approximate Orthogonalization

An ideal polar factor for a full-rank matrix is $UV^\top$ when $M = U\Sigma V^\top$. For intuition, consider a normalized matrix X_0 whose nonzero singular values lie in the convergence range of a basic Newton–Schulz polar iteration. One schematic iteration is

$$X_{k+1} = \tfrac{1}{2}X_k(3I - X_k^\top X_k), \tag{50}$$

TABLE VIII: Run record for the experiment in Fig. 9, item by item as Appendix B requires.

Item	Setting
Network	MLP 64–128–128–10 with ReLU, PyTorch default initialization
Data	scikit-learn digits (8×8 images, 10 classes); fixed split: 1,437 training, 360 held-out
Training	cross-entropy; minibatch 64; 600 updates ($\approx$27 epochs); no schedule; no weight decay
Seeds	5 per setting (initialization and batch order)
Learning rates	the same 9-point grid for every optimizer (factor $\sqrt{10}$ apart, four decades); panels (a), (b) use each optimizer's best grid point
SGD, Adam, Lion	momentum 0.9; $\beta = (0.9, 0.999)$; $\beta = (0.9, 0.99)$
Adafactor	PyTorch defaults
Shampoo	Eq. (37), $\epsilon = 10^{-4}$, exact roots every step, no grafting; biases: diagonal AdaGrad
Muon	the two hidden matrices: Nesterov momentum 0.95, five quintic Newton–Schulz steps; biases and output layer: Adam, $\eta = 2 \cdot 10^{-3}$
Sophia-H	Hutchinson estimate every 10 steps, $\gamma = 0.01$, $\rho = 1$
Precision, hardware	fp32 on a CPU (Intel Core Ultra 9 285H), PyTorch 2.11
Stopping	fixed budget of 600 updates; a non-finite loss or one above 10^3 is marked diverged
Axes	(a) gradient evaluations (a Hessian–vector product counts as two); (b) optimizer time on one CPU thread, evaluation excluded, median of three repeats
Memory	persistent optimizer-state tensors only (fp32 bytes, after two updates), including Shampoo's cached roots; weights, gradients and activations excluded

shown for a compatible matrix shape and mathematical normalization. If $X_k = U\,\mathrm{diag}(s_i)V^\top$ with fixed singular directions, each singular value evolves according to

$$s_{i,k+1} = \tfrac{1}{2} s_{i,k}(3 - s_{i,k}^2). \tag{51}$$

The fixed value $s = 1$ is attractive locally because the derivative of the scalar map is $\frac{3}{2}(1-s^2)$, which vanishes at $s = 1$. Hence errors in singular values can shrink rapidly once they are close to one. This is the basic numerical reason matrix-multiplication iterations can approximate polar factors without computing full singular-value decompositions. Practical Muon implementations use their own polynomial coefficients, scaling, iteration counts, and numerical safeguards [7], [66]; Eq. (50) is a *didactic example*, not a verbatim statement of Muon's production implementation.

An important caveat concerns zero or very small singular values. A polar factor for a rank-deficient matrix is not uniquely determined outside the supported subspace, and numerical iterations require care near zero. Furthermore, approximating the polar factor of a momentum update is an operation on *that update matrix*; it does not compute the Hessian of the neural-network loss. This is why the name "Newton–Schulz" should not cause Muon to be counted as a classical Newton optimizer.

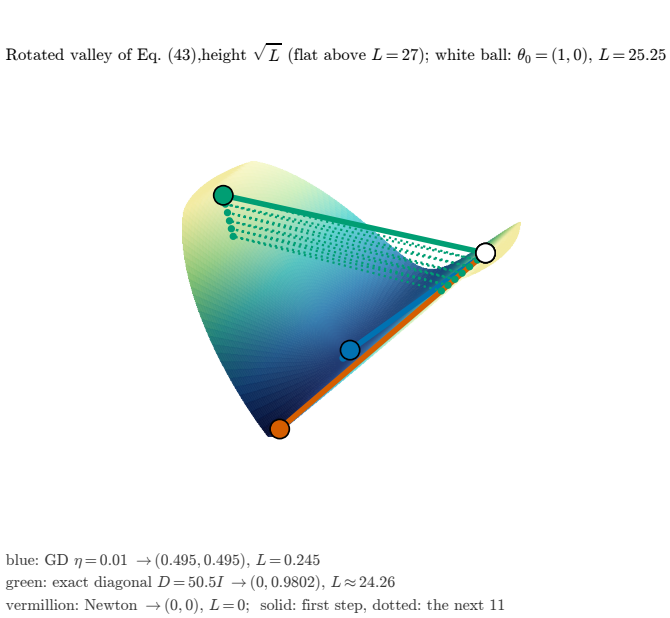


Fig. 11: The construction proposed in Appendix C-A, drawn from the 2×2 matrix of Eq. (43). All three methods start at $\theta_0 = (1, 0)$ with the same gradient $g_0 = (50.5, -49.5)^\top$. The valley is drawn in 3D (height $\sqrt{L}$). Solid lines: the first steps of Eqs. (44) and (45) and the Newton step, ending at the losses quoted in the text, 0.245, about 24.26, and 0; dotted: the next 11 steps. The exact diagonal preconditioner bounces across the valley because its multipliers are ± 0.98 (Table IX).

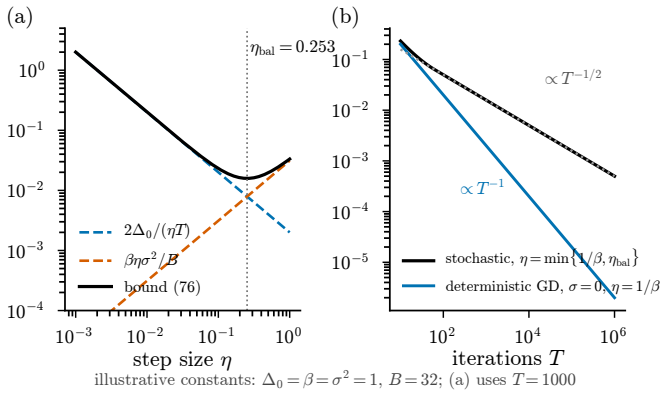


Fig. 12: The bound of Eq. (47) with illustrative constants. (a) Its two terms as functions of η: the balanced choice of Eq. (49) sits where they cross. (b) The resulting worst-case bound versus T: order $T^{-1/2}$ with noise, against the deterministic order T^{-1} of Eq. (15). These are upper bounds, not predicted training curves.

D. Deriving Natural Gradient From a Small-KL Constraint

Ordinary GD asks which Euclidean move has the greatest first-order decrease. Natural gradient instead restricts how much the model distribution may change. Near θ, approximate the KL change by $\frac{1}{2}\Delta^\top F \Delta$. Then solve

$$\min_{\Delta} g^\top \Delta \quad \text{subject to} \quad \tfrac{1}{2}\Delta^\top F \Delta \le r^2/2. \tag{52}$$

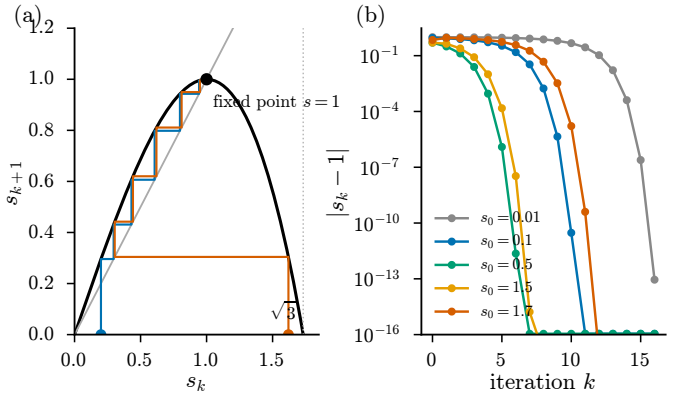


Fig. 13: The scalar Newton–Schulz map of Eq. (51). (a) Cobweb iterations from $s_0 = 0.2$ and $s_0 = 1.62$: both are drawn to the fixed point $s = 1$, where the map has zero slope. (b) Distance to one: slow growth while s_k is small (a factor near 3/2 per step), then rapid convergence once s_k is near one. Starting values must lie below $\sqrt{3}$ for this basic iteration.

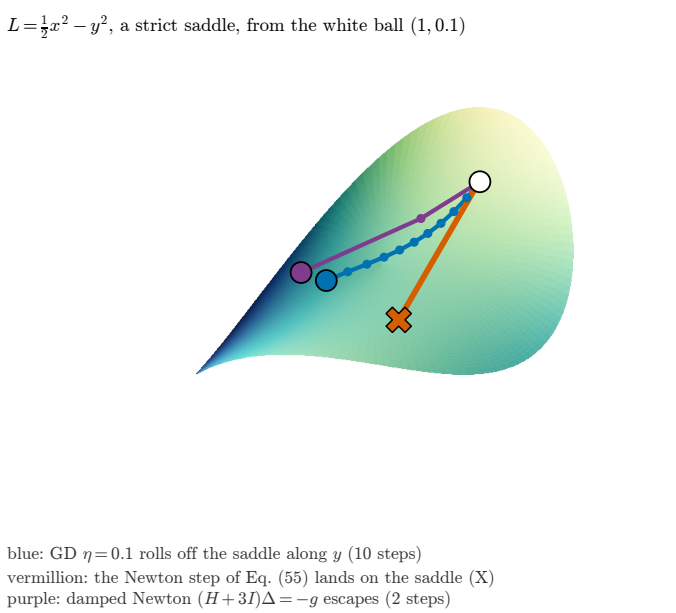


Fig. 14: The saddle of Eq. (53) from $(1, 0.1)$. GD (Eq. (54), $\eta = 0.1$) contracts x and expands y, moving away from the saddle; the unrestricted Newton step of Eq. (55) lands exactly on it. Damping as in Eq. (33) with $\lambda = 3$ makes $H + \lambda I$ positive definite, and the step again escapes along y.

When $F \succ 0$ and the constraint is active, the Lagrangian is $\mathcal{L}(\Delta, \lambda) = g^\top \Delta + \frac{\lambda}{2}(\Delta^\top F \Delta - r^2)$. Stationarity yields $g + \lambda F \Delta = 0$, so $\Delta = -\lambda^{-1} F^{-1} g$. The radius determines the multiplier λ, while the *direction* is $-F^{-1}g$. This gives an intuitive derivation of the natural gradient and shows why its goal is distinct from minimizing a Hessian quadratic model [59], [57].

Appendix D
Non-Convex Geometry: When Additional Curvature Can Mislead

A. A Saddle Where Newton Reaches the Wrong Destination

Consider the two-dimensional loss

$$L(x, y) = \frac{1}{2}x^2 - y^2, \qquad \nabla L = \begin{bmatrix} x \\ -2y \end{bmatrix}, \qquad H = \begin{bmatrix} 1 & 0 \\ 0 & -2 \end{bmatrix}. \tag{53}$$

The origin satisfies $\nabla L(0, 0) = 0$, but it is a strict saddle because the y direction has negative curvature. Start at $(x_0, y_0) = (1, 0.1)$. A GD step with learning rate η produces

$$(x_1, y_1) = ((1 - \eta)x_0, (1 + 2\eta)y_0). \tag{54}$$

The x component contracts when $0 < \eta < 2$, while the y component expands because $1 + 2\eta > 1$. In this local quadratic example, negative curvature is precisely what pushes GD away from the saddle, provided initialization has a nonzero component in that direction [8], [9].

Now take the full, unrestricted Newton step. Since $H^{-1} = \mathrm{diag}(1, -1/2)$, we obtain

$$\begin{bmatrix} x_1 \\ y_1 \end{bmatrix} = \begin{bmatrix} x_0 \\ y_0 \end{bmatrix} - H^{-1}\nabla L(x_0, y_0) = \begin{bmatrix} 0 \\ 0 \end{bmatrix}. \tag{55}$$

Newton reaches the *saddle* in one step! This is not a contradiction: Newton's equation finds a stationary point of a quadratic approximation. A nonconvex quadratic can have a stationary point that is not a local minimum. The example motivates damping, trust regions, cubic regularization, negative-curvature steps, and second-order stationarity criteria [10], [44], [45]. It also exposes why saying "second order is better because it knows the curvature" is incomplete.

The same phenomenon appears in higher dimensions. If H has both positive and negative eigenvalues, an unconstrained Newton direction $-H^{-1}g$ may point partly uphill, and the quadratic model is unbounded below along negative eigendirections. A trust-region constraint keeps the candidate displacement finite and allows an algorithm to exploit negative curvature when appropriate. But the exact trust-region solution and acceptance logic have computational costs. The correct comparison is not whether a method uses curvature, but whether it uses *the right curvature model and safeguards* for the region it encounters.

B. Stability of Preconditioned Gradient Descent

Suppose locally that the loss is the positive-definite quadratic $L(\theta) = \frac{1}{2}\theta^\top H \theta$, and apply a fixed symmetric positive-definite preconditioner M:

$$\theta_{t+1} = (I - \eta M^{-1} H)\theta_t. \tag{56}$$

Although $M^{-1}H$ need not be symmetric, it is similar to the symmetric positive-definite matrix

$$K = M^{-1/2} H M^{-1/2}. \tag{57}$$

TABLE IX: Equation (58) evaluated for three fixed preconditioners on the rotated quadratic of Eq. (43). The multipliers $1-\eta\lambda(K)$ at $\eta=1$ explain the one-step results of Appendix C-A and Fig. 11.

Preconditioner M	**Eigenvalues of** K	**Stable for**	**Multipliers at** $\eta=1$
I (GD)	1, 100	$0<\eta<0.02$	0, -99 (unstable)
$\mathrm{diag}(H)=50.5I$	0.0198, 1.980	$0<\eta<1.010$	0.980, -0.980
H (Newton)	1, 1	$0<\eta<2$	0, 0

To verify the similarity, multiply $M^{-1}H$ on the left by $M^{1/2}$ and the right by $M^{-1/2}$; the result is K. Thus $M^{-1}H$ has real positive eigenvalues equal to those of K. The preconditioned error contracts in every eigenmode if

$$\boxed{0<\eta<\frac{2}{\lambda_{\max}(K)}.} \tag{58}$$

For ordinary GD, $M=I$ and $K=H$, recovering the textbook threshold. For exact Newton preconditioning on a positive-definite quadratic, $M=H$, so $K=I$ and a unit step sends every eigencomponent to zero. For approximate or stale preconditioners, the eigenvalues of K may spread again. This calculation provides a common language for understanding conditioning and the local stability of several variable-metric methods [18], [1].

For changing stochastic preconditioners M_t, however, the analysis is more difficult: M_t may depend on noisy gradients and need not commute with the Hessian; the objective may become non-convex along the trajectory; and the relevant matrices may be singular or indefinite. One must not apply Eq. (58) as if it were an Adam, Shampoo, or K-FAC convergence theorem. It is an exact statement about a local fixed quadratic model with a fixed positive-definite preconditioner.

C. A Short Conceptual Checklist for Reading Optimizer Papers

Before accepting an optimizer's theoretical or empirical claim, identify four objects explicitly. First, the *objective class*: is the loss convex, strongly convex, smooth non-convex, or a particular network architecture? Second, the *information oracle*: does each update require gradients, Hessian–vector products, factor inverses, extra passes, or a full linear solve? Third, the *success criterion*: training loss, gradient norm, first- or second-order stationarity, validation accuracy, token throughput, or total compute? Fourth, the *operating regime*: minibatch size, precision, model shape, accelerator, damping, and hyperparameter-search budget. These objects determine whether two results are comparable.

For example, a proof of fast local Newton convergence and a report that Muon trains a transformer faster than AdamW do not conflict: one concerns a regular local minimizer and mathematical iteration error, while the other concerns a concrete stochastic implementation, model architecture, and hardware-dependent cost. Similarly, a convex regret guarantee for Shampoo and a non-convex stationarity result for SGD answer different mathematical questions. The role of a rigorous survey is not to declare a winner from these dissimilar statements but to explain what the statements do and do not establish [3], [6], [7].

References

[1] J. Nocedal and S. Wright, "Numerical Optimization, 2nd ed.," Springer, 2006. [Online]. Available: https://doi.org/10.1007/978-0-387-40065-5.

[2] L. Bottou, "Large-scale machine learning with stochastic gradient descent," Proceedings of COMPSTAT, pp. 177–186, 2010. [Online]. Available: https://doi.org/10.1007/978-3-7908-2604-3_16.

[3] L. Bottou, F. E. Curtis, and J. Nocedal, "Optimization methods for large-scale machine learning," SIAM Review, vol. 60, pp. 223–311, 2018. [Online]. Available: https://doi.org/10.1137/16M1080173.

[4] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," Proceedings of ICLR, 2015. [Online]. Available: https://arxiv.org/abs/1412.6980.

[5] N. Shazeer and M. Stern, "Adafactor: Adaptive learning rates with sublinear memory cost," Proceedings of ICML, PMLR 80, pp. 4596–4604, 2018. [Online]. Available: https://proceedings.mlr.press/v80/shazeer18a.html.

[6] V. Gupta, T. Koren, and Y. Singer, "Shampoo: Preconditioned stochastic tensor optimization," Proceedings of ICML, PMLR 80, 2018. [Online]. Available: https://proceedings.mlr.press/v80/gupta18a.html.

[7] K. Jordan et al., "Muon: An optimizer for hidden layers in neural networks," Technical article and open-source implementation, 2024. [Online]. Available: https://kellerjordan.github.io/posts/muon/.

[8] Y. N. Dauphin et al., "Identifying and attacking the saddle point problem in high-dimensional non-convex optimization," Advances in Neural Information Processing Systems, 2014. [Online]. Available: https://arxiv.org/abs/1406.2572.

[9] J. D. Lee, M. Simchowitz, M. I. Jordan, and B. Recht, "Gradient descent only converges to minimizers," Proceedings of COLT, PMLR 49, 2016. [Online]. Available: https://proceedings.mlr.press/v49/lee16.html.

[10] C. Jin et al., "How to escape saddle points efficiently," Proceedings of ICML, PMLR 70, 2017. [Online]. Available: https://proceedings.mlr.press/v70/jin17a.html.

[11] X. Chen et al., "Symbolic discovery of optimization algorithms," Advances in Neural Information Processing Systems, vol. 36, 2023. [Online]. Available: https://arxiv.org/abs/2302.06675.

[12] H. Liu, Z. Li, D. Hall, P. Liang, and T. Ma, "Sophia: A scalable stochastic second-order optimizer for language model pre-training," Proceedings of ICLR, 2024. [Online]. Available: https://arxiv.org/abs/2305.14342.

[13] N. Vyas et al., "SOAP: Improving and stabilizing Shampoo using Adam," arXiv:2409.11321, 2024. [Online]. Available: https://arxiv.org/abs/2409.11321.

[14] X. Glorot and Y. Bengio, "Understanding the difficulty of training deep feedforward neural networks," Proceedings of AISTATS, PMLR 9, 2010. [Online]. Available: https://proceedings.mlr.press/v9/glorot10a.html.

[15] A. C. Wilson, R. Roelofs, M. Stern, N. Srebro, and B. Recht, "The marginal value of adaptive gradient methods in machine learning," Advances in Neural Information Processing Systems, 2017. [Online]. Available: https://arxiv.org/abs/1705.08292.

[16] N. S. Keskar et al., "On large-batch training for deep learning: Generalization gap and sharp minima," Proceedings of ICLR, 2017. [Online]. Available: https://arxiv.org/abs/1609.04836.

[17] L. Dinh, R. Pascanu, S. Bengio, and Y. Bengio, "Sharp minima can generalize for deep nets," Proceedings of ICML, PMLR 70, 2017. [Online]. Available: https://proceedings.mlr.press/v70/dinh17b.html.

[18] Y. Nesterov, "Introductory Lectures on Convex Optimization: A Basic Course," Kluwer Academic Publishers, 2004. [Online]. Available: https://doi.org/10.1007/978-1-4419-8853-9.

[19] R. Ge, F. Huang, C. Jin, and Y. Yuan, "Escaping from saddle points—Online stochastic gradient for tensor decomposition," Proceedings of COLT, PMLR 40, 2015. [Online]. Available: https://proceedings.mlr.press/v40/Ge15.html.

[20] H. Robbins and S. Monro, "A stochastic approximation method," Annals of Mathematical Statistics, vol. 22, pp. 400–407, 1951. [Online]. Available: https://doi.org/10.1214/aoms/1177729586.

[21] J. Kiefer and J. Wolfowitz, "Stochastic estimation of the maximum of a regression function," Annals of Mathematical Statistics, vol. 23, pp. 462–466, 1952. [Online]. Available: https://doi.org/10.1214/aoms/1177729392.

[22] A. Nemirovski, A. Juditsky, G. Lan, and A. Shapiro, "Robust stochastic approximation approach to stochastic programming," SIAM Journal on Optimization, vol. 19, pp. 1574–1609, 2009. [Online]. Available: https://doi.org/10.1137/070704277.

[23] P. Goyal et al., "Accurate, large minibatch SGD: Training ImageNet in 1 hour," arXiv:1706.02677, 2017. [Online]. Available: https://arxiv.org/abs/1706.02677.

[24] P. Micikevicius et al., "Mixed precision training," Proceedings of ICLR, 2018. [Online]. Available: https://arxiv.org/abs/1710.03740.

[25] S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He, "ZeRO: Memory optimizations toward training trillion parameter models," Proceedings of SC20, 2020. [Online]. Available: https://arxiv.org/abs/1910.02054.

[26] S. Ghadimi and G. Lan, "Stochastic first- and zeroth-order methods for nonconvex stochastic programming," SIAM Journal on Optimization, vol. 23, pp. 2341–2368, 2013. [Online]. Available: https://doi.org/10.1137/120880811.

[27] R. Ward, X. Wu, and L. Bottou, "AdaGrad stepsizes: Sharp convergence over nonconvex landscapes," Journal of Machine Learning Research, vol. 21, 2020. [Online]. Available: https://jmlr.org/papers/v21/18-808.html.

[28] Y. Nesterov, "A method of solving a convex programming problem with convergence rate $O(1/k^2)$," Soviet Mathematics Doklady, vol. 27, pp. 372–376, 1983.

[29] J. Duchi, E. Hazan, and Y. Singer, "Adaptive subgradient methods for online learning and stochastic optimization," Journal of Machine Learning Research, vol. 12, pp. 2121–2159, 2011. [Online]. Available: https://jmlr.org/papers/v12/duchi11a.html.

[30] T. Tieleman and G. Hinton, "Lecture 6.5—RMSProp: Divide the gradient by a running average of its recent magnitude," Neural Networks for Machine Learning, Coursera lecture, 2012.

[31] M. D. Zeiler, "ADADELTA: An adaptive learning rate method," arXiv:1212.5701, 2012. [Online]. Available: https://arxiv.org/abs/1212.5701.

[32] S. J. Reddi, S. Kale, and S. Kumar, "On the convergence of Adam and beyond," Proceedings of ICLR, 2018. [Online]. Available: https://openreview.net/forum?id=ryQu7f-RZ.

[33] X. Chen, S. Liu, R. Sun, and M. Hong, "On the convergence of a class of Adam-type algorithms for non-convex optimization," Proceedings of ICLR, 2019. [Online]. Available: https://arxiv.org/abs/1808.02941.

[34] M. Zaheer, S. Reddi, D. Sachan, S. Kale, and S. Kumar, "Adaptive methods for nonconvex optimization," Advances in Neural Information Processing Systems, 2018. [Online]. Available: https://arxiv.org/abs/1806.09318.

[35] A. D'efossez, L. Bottou, F. Bach, and N. Usunier, "A simple convergence proof of Adam and Adagrad," Transactions on Machine Learning Research, 2022. [Online]. Available: https://arxiv.org/abs/2003.02395.

[36] L. Liu et al., "On the variance of the adaptive learning rate and beyond," Proceedings of ICLR, 2020. [Online]. Available: https://arxiv.org/abs/1908.03265.

[37] J. Zhuang et al., "AdaBelief optimizer: Adapting stepsizes by the belief in observed gradients," Advances in Neural Information Processing Systems, 2020. [Online]. Available: https://arxiv.org/abs/2010.07468.

[38] L. Luo, Y. Xiong, Y. Liu, and X. Sun, "Adaptive gradient methods with dynamic bound of learning rate," Proceedings of ICLR, 2019. [Online]. Available: https://arxiv.org/abs/1902.09843.

[39] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," Proceedings of ICLR, 2019. [Online]. Available: https://arxiv.org/abs/1711.05101.

[40] A. Defazio and K. Mishchenko, "Learning-rate-free learning by D-adaptation," Proceedings of ICML, PMLR 202, 2023. [Online]. Available: https://proceedings.mlr.press/v202/defazio23a.html.

[41] A. Defazio et al., "The road less scheduled," Advances in Neural Information Processing Systems, vol. 37, 2024. [Online]. Available: https://arxiv.org/abs/2405.15682.

[42] M. R. Zhang et al., "Lookahead optimizer: k steps forward, 1 step back," Advances in Neural Information Processing Systems, 2019. [Online]. Available: https://arxiv.org/abs/1907.08610.

[43] A. R. Conn, N. I. M. Gould, and P. L. Toint, "Trust Region Methods," SIAM, 2000. [Online]. Available: https://doi.org/10.1137/1.9780898719857.

[44] Y. Nesterov and B. T. Polyak, "Cubic regularization of Newton method and its global performance," Mathematical Programming, vol. 108, pp. 177–205, 2006. [Online]. Available: https://doi.org/10.1007/s10107-006-0706-8.

[45] C. Cartis, N. I. M. Gould, and P. L. Toint, "Adaptive cubic regularisation methods for unconstrained optimization. Part I: Motivation, convergence and numerical results," Mathematical Programming, vol. 127, pp. 245–295, 2011. [Online]. Available: https://doi.org/10.1007/s10107-009-0286-5.

[46] C. G. Broyden, "The convergence of a class of double-rank minimization algorithms: 2. The new algorithm," IMA Journal of Applied Mathematics, vol. 6, pp. 222–231, 1970. [Online]. Available: https://doi.org/10.1093/imamat/6.3.222.

[47] R. Fletcher, "A new approach to variable metric algorithms," The Computer Journal, vol. 13, pp. 317–322, 1970. [Online]. Available: https://doi.org/10.1093/comjnl/13.3.317.

[48] D. Goldfarb, "A family of variable-metric methods derived by variational means," Mathematics of Computation, vol. 24, pp. 23–26, 1970. [Online]. Available: https://doi.org/10.1090/S0025-5718-1970-0258249-6.

[49] D. F. Shanno, "Conditioning of quasi-Newton methods for function minimization," Mathematics of Computation, vol. 24, pp. 647–656, 1970. [Online]. Available: https://doi.org/10.1090/S0025-5718-1970-0274029-X.

[50] J. Nocedal, "Updating quasi-Newton matrices with limited storage," Mathematics of Computation, vol. 35, pp. 773–782, 1980. [Online]. Available: https://doi.org/10.1090/S0025-5718-1980-0572855-7.

[51] D. C. Liu and J. Nocedal, "On the limited memory BFGS method for large scale optimization," Mathematical Programming, vol. 45, pp. 503–528, 1989. [Online]. Available: https://doi.org/10.1007/BF01589116.

[52] B. A. Pearlmutter, "Fast exact multiplication by the Hessian," Neural Computation, vol. 6, pp. 147–160, 1994. [Online]. Available: https://doi.org/10.1162/neco.1994.6.1.147.

[53] N. N. Schraudolph, "Fast curvature matrix-vector products for second-order gradient descent," Neural Computation, vol. 14, pp. 1723–1738, 2002. [Online]. Available: https://doi.org/10.1162/08997660260028683.

[54] J. Martens, "Deep learning via Hessian-free optimization," Proceedings of ICML, 2010. [Online]. Available: https://icml.cc/Conferences/2010/papers/458.pdf.

[55] J. Martens and I. Sutskever, "Learning recurrent neural networks with Hessian-free optimization," Proceedings of ICML, 2011. [Online]. Available: https://www.cs.toronto.edu/~jmartens/docs/HF-RNN.pdf.

[56] A. Botev, H. Ritter, and D. Barber, "Practical Gauss–Newton optimisation for deep learning," Proceedings of ICML, PMLR 70, 2017. [Online]. Available: https://proceedings.mlr.press/v70/botev17a.html.

[57] J. Martens, "New insights and perspectives on the natural gradient method," Journal of Machine Learning Research, vol. 21, 2020. [Online]. Available: https://jmlr.org/papers/v21/17-678.html.

[58] F. Kunstner, L. Balles, and P. Hennig, "Limitations of the empirical Fisher approximation for natural gradient descent," Advances in Neural Information Processing Systems, 2019. [Online]. Available: https://arxiv.org/abs/1905.12558.

[59] S.-I. Amari, "Natural gradient works efficiently in learning," Neural Computation, vol. 10, pp. 251–276, 1998. [Online]. Available: https://doi.org/10.1162/089976698300017746.

[60] R. Pascanu and Y. Bengio, "Revisiting natural gradient for deep networks," Proceedings of ICLR, 2014. [Online]. Available: https://arxiv.org/abs/1301.3584.

[61] J. Martens and R. Grosse, "Optimizing neural networks with Kronecker-factored approximate curvature," Proceedings of ICML, PMLR 37, 2015. [Online]. Available: https://proceedings.mlr.press/v37/martens15.html.
[62] R. Grosse and J. Martens, "A Kronecker-factored approximate Fisher matrix for convolution layers," Proceedings of ICML, PMLR 48, 2016. [Online]. Available: https://proceedings.mlr.press/v48/grosse16.html.
[63] R. Anil et al., "Scalable second order optimization for deep learning," arXiv:2002.09018, 2020. [Online]. Available: https://arxiv.org/abs/2002.09018.
[64] Z. Yao et al., "AdaHessian: An adaptive second order optimizer for machine learning," Proceedings of AAAI, 2021. [Online]. Available: https://arxiv.org/abs/2006.00719.
[65] X. Ma, "Apollo: An adaptive parameter-wise diagonal quasi-Newton method for nonconvex stochastic optimization," Proceedings of AAAI, 2021. [Online]. Available: https://arxiv.org/abs/2009.13586.
[66] J. Bernstein and L. Newhouse, "Old optimizer, new norm: An anthology," arXiv:2409.20325, 2024. [Online]. Available: https://arxiv.org/abs/2409.20325.
[67] J. Hoffmann et al., "Training compute-optimal large language models," arXiv:2203.15556, 2022. [Online]. Available: https://arxiv.org/abs/2203.15556.
[68] J. Kaplan et al., "Scaling laws for neural language models," arXiv:2001.08361, 2020. [Online]. Available: https://arxiv.org/abs/2001.08361.
[69] J. M. Cohen, S. Kaur, Y. Li, J. Z. Kolter, and A. Talwalkar, "Gradient descent on neural networks typically occurs at the edge of stability," Proceedings of ICLR, 2021. [Online]. Available: https://arxiv.org/abs/2103.00065.
[70] N. S. Keskar and R. Socher, "Improving generalization performance by switching from Adam to SGD," arXiv:1712.07628, 2017. [Online]. Available: https://arxiv.org/abs/1712.07628.
[71] P. Foret, A. Kleiner, H. Mobahi, and B. Neyshabur, "Sharpness-aware minimization for efficiently improving generalization," Proceedings of ICLR, 2021. [Online]. Available: https://arxiv.org/abs/2010.01412.
[72] J. L. Ba, J. R. Kiros, and G. E. Hinton, "Layer normalization," arXiv:1607.06450, 2016. [Online]. Available: https://arxiv.org/abs/1607.06450.